

România
Ministerul Apărării Naționale
Academia Tehnică Militară "Ferdinand I"

Facultatea de Sisteme Informactice și Securitate Cibernetică
Specializarea Absolvită



Soluție inteligentă de analiză a înregistrărilor video

Coordonator Științific

Grad Prenume NUME

Absolvent

Grad Dumitru Estera

Conține _____ file
Inventariat sub numărul _____
Cu poziția din indicator _____
Cu termen de păstrare _____

București
2026

Mulțumiri pentru persoanele care au sprijinit procesul de realizare a acestei lucrări

Referatul Coordonatorului Științific

Referatul reprezintă un text în care coordonatorul științific sumarizează, ulterior finalizării conținutului efectiv, efortul pe care l-ați depus și trage concluzii cu privire la gradul de realizare a obiectivelor propuse inițial (cele prezentate în cadrul detalierii). De regulă, acest referat nu depășește cele 4 pagini alocate în acest document.

NECLASIFICAT

NECLASIFICAT

NECLASIFICAT

NECLASIFICAT

NECLASIFICAT

NECLASIFICAT

Tema Proiectului de Diplomă

Tema proiectului de diplomă (sau detalierea) reprezintă un text realizat de coordonatorul științific, în lunile următoare propunerii titlului proiectului de diplomă către facultate, în care detaliază nevoia reală a implementării unui astfel de proiect și modul în care se dorește a fi implementat. În plus, poate prezenta structura pe capitole a viitoarei lucrări, anexele ce vor fi incluse și sursele bibliografice din care studentul se va informa. De regulă, această detaliere nu depășește cele 4 pagini alocate în acest document.

NECLASIFICAT

NECLASIFICAT

NECLASIFICAT

NECLASIFICAT

NECLASIFICAT

NECLASIFICAT

Abstract

This is the abstract of the thesis, written after finishing all the other components of the document (especially the chapters). It broadly presents the objectives and the achievements of the thesis. It is present in both Romanian and English.

Rezumat

Acesta este rezumatul lucrării, realizat după finalizarea tuturor celorlaltor componente ale documentului (în special capitolele). El prezintă, în linii mari, obiectivele și realizările lucrării. Este prezent atât în limba română, cât și în cea engleză.

Cuprins

1	Introducere	1
1.1	Contextul actual al supravegherii video	1
1.2	Motivația alegerii temei	1
1.3	Problema abordată	1
1.4	Obiectivele lucrării	1
1.5	Contribuții personale	1
1.6	Structura lucrării	1
2	Stadiul Actual al Sistemelor Inteligente de Supraveghere Video	2
2.1	Evoluția sistemelor de supraveghere video	2
2.2	Soluții comerciale existente	2
2.3	Soluții open-source și academice	2
2.4	Stadiul cercetării în domeniile relevante	2
2.5	Plasarea proiectului în context	2
3	Fundamente Teoretice	3
3.1	Învățarea profundă pentru viziune artificială	4
3.1.1	Rețele neuronale convoluționale (CNN)	4
3.1.2	Arhitecturi de referință	4
3.1.3	Funcții de pierdere și regularizare	4
3.2	Detecția obiectelor	4
3.2.1	Familia YOLO	4
3.2.2	YOLOv8	4
3.2.3	YOLOv10	4
3.2.4	Metrici de evaluare	4
3.3	Recunoașterea facială	4
3.3.1	Detecția fețelor	4
3.3.2	Embeddings faciale	4
3.3.3	Funcția de pierdere ArcFace	4
3.3.4	Arhitectura MobileFaceNet (w600k_mbf)	4
3.3.5	InsightFace și pachetul buffalo_s	4
3.4	Analiza demografică și a emoțiilor	4
3.4.1	Recunoașterea expresiilor faciale	4
3.4.2	Mini-Xception	4

3.5	Recunoașterea optică a caracterelor	4
3.5.1	Detectia zonelor de text – CRAFT	4
3.5.2	Recunoașterea caracterelor – CRNN	4
3.6	Recunoașterea acțiunilor în videoclipuri	4
3.6.1	Provocările analizei temporale	4
3.6.2	Arhitectura SlowFast	4
3.6.3	PyTorchVideo	4
3.7	Procesarea imaginilor – tehnici clasice utilizate	4
3.8	Căutarea în spații vectoriale	4
3.8.1	Biblioteca FAISS	4
3.9	Tehnologii utilizate pentru aplicația web	4
3.9.1	FastAPI	4
3.9.2	Protocolul WebSocket	4
3.9.3	React și ecosistemul frontend	4
3.9.4	Codificarea Base64 a cadrelor video	4
4	Analiza Cerințelor și Proiectarea Sistemului	5
4.1	Analiza cerințelor	5
4.1.1	Cerințe funcționale	5
4.1.2	Cerințe non-funcționale	7
4.2	Cazuri de utilizare	9
4.3	Arhitectura generală a sistemului	12
4.3.1	Modelul client-server	13
4.3.1.1	Clientul	13
4.3.1.2	Serverul	13
4.3.1.3	Sursele video	14
4.3.1.4	Persistența	14
4.3.2	Diagrama de componente	14
4.3.3	Diagrama de implementare (deployment)	16
4.3.3.1	Nodul client (browser)	16
4.3.3.2	Nodul server cu GPU	16
4.3.3.3	Nodul bază de date	17
4.3.3.4	Camere IP	17
4.3.3.5	Storage (opțional)	17
4.4	Proiectarea pipeline-ului multi-threaded	17
4.4.1	Cele 7 fire de execuție	18
4.4.1.1	Firul 1 – Captura video	18
4.4.1.2	Firul 2 – Recunoaștere facială și analiză demo- grafică	19
4.4.1.3	Firul 3 – Plăcuțe de înmatriculare	19
4.4.1.4	Firul 4 – Foc și fum	19
4.4.1.5	Firul 5 – Recunoaștere acțiuni umane (HAR)	19
4.4.1.6	Firul 6 – Arme	19
4.4.1.7	Firul 7 – Fuziune	19
4.4.1.8	Justificarea separării	20
4.4.2	Mecanismele de sincronizare	20

4.4.2.1	Cozi thread-safe	20
4.4.2.2	Partajarea cadrului curent	21
4.4.2.3	Mecanismul <i>ultimul cadru disponibil</i>	21
4.4.2.4	Stare partajată protejată prin lock-uri	21
4.4.3	Justificarea procesării paralele	21
4.4.3.1	Utilizarea GPU-ului	21
4.4.3.2	Latența percepută per modul	22
4.4.3.3	Dezactivarea selectivă	22
4.4.3.4	Tolerare la eșec	22
4.4.3.5	De ce nu microservicii separate	22
4.5	Proiectarea API-ului REST	23
4.5.0.1	Convenții de denumire	23
4.5.0.2	Catalogul endpoint-urilor pe categorii	23
4.5.0.3	Structura răspunsurilor	25
4.5.0.4	Coduri de eroare	26
4.6	Proiectarea canalului WebSocket	26
4.6.0.1	Endpoint dedicat pentru streaming	26
4.6.0.2	Formatul mesajelor	27
4.6.0.3	Cadența de transmitere	28
4.6.0.4	Heartbeat și detectarea deconectării	28
4.6.0.5	Reconectare	28
4.7	Proiectarea bazei de date	29
4.7.0.1	Alegerea sistemului de baze de date	29
4.7.0.2	Diagrama entitate-relație	29
4.7.0.3	Tabelele principale	29
4.7.0.4	Chei primare și străine	31
4.7.0.5	Indexarea	31
4.8	Proiectarea sistemului de control al accesului	32
4.8.0.1	Modelul RBAC	32
4.8.0.2	Matricea de permisiuni	32
4.8.0.3	Hashing-ul parolelor	33
4.8.0.4	Gestiunea sesiunii (JWT)	33
4.9	Proiectarea logicii de alarmare	34
4.9.0.1	Reguli de declanșare	34
4.9.0.2	Captura asociată (snapshot)	35
4.9.0.3	Niveluri de severitate și mapping pe tipuri de eveniment	35
4.9.0.4	Deduplicarea temporală	36
4.10	Proiectarea interfeței utilizator	37
4.10.0.1	Structura de navigare	37
4.10.0.2	Wireframe-uri pentru ecranele principale	37
4.10.0.3	Principii UX	38
4.10.0.4	Limitări identificate față de cerințe	39
5	Implementarea Modulelor de Inteligență Artificială	40
5.1	Modulul de recunoaștere facială	40

5.1.1	Detecția fețelor și extragerea embeddings-urilor	40
5.1.1.0.1	Configurarea execuției pe GPU.	41
5.1.1.0.2	Gestionarea bibliotecilor CUDA și a TensorFlow.	41
5.1.1.0.3	Înregistrarea persoanelor.	41
5.1.2	Indexarea cu FAISS FlatL2	42
5.1.2.0.1	Maparea identităților.	42
5.1.2.0.2	Persistența și reconstrucția indexului.	42
5.1.2.0.3	Siguranța accesului concurent.	42
5.1.3	Strategia de matching	43
5.1.3.0.1	De la distanța L2 la similaritatea cosinus.	43
5.1.4	Optimizarea: omiterea analizei demografice pentru fețe cunoscute	43
5.2	Analiza demografică și emoțională	44
5.2.1	Estimarea vârstei și genului	44
5.2.2	Recunoașterea emoțiilor cu Mini-Xception	45
5.2.3	Preprocesarea crop-ului facial	45
5.2.4	Inferența pe CPU	46
5.3	Modulul de recunoaștere a plăcuțelor de înmatriculare	46
5.3.1	Detecția plăcuțelor cu YOLOv8	47
5.3.2	Up-scaling-ul plăcuțelor mici	47
5.3.3	Pipeline-ul de preprocesare pentru OCR	47
5.3.4	Apelul EasyOCR	48
5.3.5	Fallback pe imaginea color	48
5.3.6	Tracking-ul plăcuțelor între cadre	48
5.3.7	Validarea formatului	49
5.3.8	Votarea temporală	49
5.4	Modulul de detecție foc și fum	49
5.4.1	Modelul YOLOv10 pre-antrenat	50
5.4.2	Pipeline-ul de filtrare în cinci niveluri	50
5.4.2.1	Praguri de încredere per clasă	50
5.4.2.2	Filtre de dimensiune	50
5.4.2.3	Verificarea cromatică HSV	51
5.4.2.4	Confirmarea temporală prin tracking IoU	51
5.4.2.5	Cooldown de alertă pe locație	51
5.4.3	Flag-urile <i>confirmed</i> și <i>alert</i>	52
5.5	Modulul de detecție a armelor (model fine-tuned propriu)	52
5.5.1	Justificarea unei singure clase <i>weapon</i>	52
5.5.2	Selectarea modelului de bază	53
5.5.3	Dataseturile utilizate	53
5.5.4	Preprocesarea și unificarea	53
5.5.5	Strategia de denumire (<i>zen_</i> / <i>rf_</i>)	54
5.5.6	Parametrii de antrenare	54
5.5.7	Augmentările folosite	54
5.5.8	Rezultatele antrenării	55
5.5.9	Discuție asupra rezultatelor	55

5.6	Modulul de recunoaștere a acțiunilor umane (fine-tuning SlowFast)	56
5.6.1	Justificarea alegerii SlowFast R50	56
5.6.2	Construcția setului de date	57
5.6.2.1	Clasa <i>normal</i>	57
5.6.2.2	Clasa <i>fight</i>	57
5.6.2.3	Clasa <i>vandalism</i> – dataset propriu	57
5.6.2.3.1	Justificarea creării unui dataset propriu.	57
5.6.2.3.2	Etapă 1: Scraping automat YouTube.	57
5.6.2.3.3	Etapă 2: Filtrare manuală.	57
5.6.2.3.4	Etapă 3: Editare în OpenShot.	58
5.6.2.3.5	Setul final.	58
5.6.3	Pregătirea modelului pentru fine-tuning	58
5.6.3.0.1	Formatul intrării (contractul SlowFast).	58
5.6.4	Parametrii de antrenare	58
5.6.5	Rezultatele	59
5.6.6	Analiza per clasă	59
5.6.6.0.1	Integrarea în aplicație.	60
5.7	Fuziunea rezultatelor pe cadru	60
5.7.1	Dispecerizarea cadrelor și procesarea în paralel	61
5.7.2	Sincronizarea rezultatelor după <i>frame_id</i>	61
5.7.3	Compunerea cadrului adnotat	61
5.7.4	Generarea alarmelor și deduplicarea	62
5.7.5	Difuzarea către clienți prin WebSocket	62
6	Implementarea Aplicației Web	63
6.1	Structura backend-ului FastAPI	64
6.2	Endpoint-ul WebSocket pentru streaming live	64
6.3	Modelul de date și persistența	64
6.4	Sistemul de autentificare	64
6.5	Modulele frontend-ului React	64
6.5.1	Autentificare	64
6.5.2	Vizualizarea live a camerelor (Dashboard)	64
6.5.3	Activarea/Dezactivarea modulelor AI	64
6.5.4	Gestiunea persoanelor (CRUD)	64
6.5.5	Gestiunea zonelor	64
6.5.6	Gestiunea camerelor	64
6.5.7	Gestiunea plăcuțelor de înmatriculare	64
6.5.8	Gestiunea alarmelor	64
6.5.9	Jurnale de activitate	64
6.5.10	Statistici și KPI-uri	64
6.5.11	Gestiunea utilizatorilor	64
6.6	Logica de business pentru declanșarea alertelor	64
6.7	Sincronizarea camerei conectate dinamic cu registrul de camere	64
7	Testarea și Evaluarea Sistemului	65
7.1	Metodologia de testare	65

7.2	Configurația hardware folosită	65
7.3	Evaluarea pipeline-ului integrat	65
7.3.1	FPS la procesare concurentă	65
7.3.2	Utilizarea memoriei GPU și RAM	65
7.3.3	Latența end-to-end	65
7.4	Scenarii de test funcțional	65
7.5	Discuție asupra false pozitivelor și false negativelor	65
7.6	Limitări observate	65
8	Concluzii și Direcții de Dezvoltare	66
8.1	Sinteza rezultatelor	66
8.2	Obiective atinse vs. obiective propuse	66
8.3	Contribuții originale	66
8.4	Limitări ale soluției actuale	66
8.5	Direcții de dezvoltare viitoare	66
	Bibliografie	67

Listă de figuri

NECLASIFICAT

NECLASIFICAT

Listă de Abrevieri

AI	<i>Artificial Intelligence</i>
API	<i>Application Programming Interface</i>
CNN	<i>Convolutional Neural Network</i>
CPU	<i>Central Processing Unit</i>
CRNN	<i>Convolutional Recurrent Neural Network</i>
FAISS	<i>Facebook AI Similarity Search</i>
FPS	<i>Frames Per Second</i>
GPU	<i>Graphics Processing Unit</i>
HSV	<i>Hue, Saturation, Value</i>
IoU	<i>Intersection over Union</i>
JWT	<i>JSON Web Token</i>
mAP	<i>mean Average Precision</i>
OCR	<i>Optical Character Recognition</i>
ONNX	<i>Open Neural Network Exchange</i>
RBAC	<i>Role-Based Access Control</i>
REST	<i>Representational State Transfer</i>
YOLO	<i>You Only Look Once</i>

NECLASIFICAT

NECLASIFICAT

Capitolul 1: Introducere

- 1.1 Contextul actual al supravegherii video
- 1.2 Motivația alegerii temei
- 1.3 Problema abordată
- 1.4 Obiectivele lucrării
- 1.5 Contribuții personale
- 1.6 Structura lucrării

Capitolul 2: Stadiul Actual al Sistemelor Inteligente de Supraveghere Video

- 2.1 Evoluția sistemelor de supraveghere video
- 2.2 Soluții comerciale existente
- 2.3 Soluții open-source și academice
- 2.4 Stadiul cercetării în domeniile relevante
- 2.5 Plasarea proiectului în context

NECLASIFICAT

Capitolul 3: Fundamente Teoretice

3.1 Învățarea profundă pentru viziune artificială

3.1.1 Rețele neuronale convoluționale (CNN)

3.1.2 Arhitecturi de referință

3.1.3 Funcții de pierdere și regularizare

3.2 Detectia obiectelor

3.2.1 Familia YOLO

3.2.2 YOLOv8

3.2.3 YOLOv10

3.2.4 Metrice de evaluare

3.3 Recunoașterea facială

3.3.1 Detectia fețelor

3.3.2 Embeddings faciale

3.3.3 Funcția de pierdere ArcFace

3.3.4 Arhitectura MobileFaceNet (w600k_mbf)

3.3.5 InsightFace și pachetul buffalo_s

3.4 Analiza demografică și a emoțiilor

3.4.1 Recunoașterea expresiilor faciale

3.4.2 Mini-Xception

3.5 Recunoașterea optică a caracterelor

3.5.1 Detectia zonelor de text – CRAFT

3.5.2 Recunoașterea caracterelor – CRNN

3.6 Recunoașterea acțiunilor în videoclipuri

3.6.1 Recunoașterea acțiunilor în videoclipuri

Capitolul 4: Analiza Cerințelor și Proiectarea Sistemului

4.1 Analiza cerințelor

Înainte de prezentarea arhitecturii și a detaliilor de implementare, este important să fie stabilit în mod clar atât ceea ce trebuie să realizeze sistemul, cât și modul în care acesta trebuie să își îndeplinească funcțiile. În acest sens, secțiunea de față definește cerințele funcționale, care descriu capabilitățile oferite utilizatorului, precum și cerințele nefuncționale, care stabilesc constrângerile și criteriile de calitate ale soluției. Aceste cerințe vor constitui baza pe care se vor fundamenta deciziile de proiectare prezentate în capitolele următoare..

Distincția dintre cele două categorii este esențială: cerințele funcționale descriu comportamentul observabil al sistemului în timp ce cerințele nefuncționale fixează pragurile de performanță, securitate și uzabilitate sub care soluția nu mai este considerată viabilă pentru scenariul de supraveghere vizat. Ambele au fost derivate plecând de la analiza stadiului actual (Capitolul 2) și de la limitările identificate în soluțiile comerciale existente.

4.1.1 Cerințe funcționale

Cerințele funcționale sunt prezentate sub formă de listă numerotată, fiecare identificator **F_x** fiind referit ulterior în capitolele de proiectare și implementare. Lista nu detaliază toate operațiile CRUD, ci organizează funcționalitățile pe domenii coerente.

F1 – Streaming live multi-cameră. Sistemul trebuie să accepte simultan mai multe surse video (incluzând camera locală a stației, de tip webcam USB prin `cv2.VideoCapture`, precum și camere IP adăugate dinamic prin URL de flux RTSP/HTTP/MJPEG) pentru a livra fluxurile procesate către clienții web în timp real, folosind canalul WebSocket dedicat. Fiecare cameră are un identificator logic unic (**CAM-01**, **CAM-02**, ...) și poate fi pornită, oprită sau adăugată/eliminată fără a opri restul sistemului.

- F2 – Recunoaștere facială cu autorizare pe zone.** Pentru fiecare cadru, sistemul detectează fețele prezente, calculează embedding-uri faciale și le compară cu indexul FAISS construit din baza de persoane înregistrate. Pe lângă identificarea propriu-zisă, sistemul verifică dacă persoana are dreptul de acces în zona asociată camerei curente, marcând vizual indivizii neautorizați. Pentru a îmbunătăți robustețea recunoașterii, o persoană poate avea înregistrate mai multe imagini faciale. În plus, ștergerea unui utilizator declanșează reconstrucția indexului FAISS, iar modificarea metadatelor (nume, departament, zone autorizate) duce la reîmprospătarea cache-urilor de autorizare.
- F2bis – Extracție de attribute demografice.** Pe lângă identificare, sistemul estimează attribute demografice ale fețelor detectate (vârstă, gen și emoție) folosite atât pentru statisticile agregate (F10), cât și pentru îmbogățirea metadatelor jurnalizate per detecție.
- F3 – Detecție și recunoaștere a plăcuțelor de înmatriculare.** Sistemul localizează plăcuțele de înmatriculare în cadrele video și extrage textul acestora prin OCR, cu stabilizarea citirii prin vot pe mai multe cadre. O plăcuță este considerată autorizată dacă numărul stabilizat este regăsit în baza de date de plăcuțe înregistrate.
- F4 – Detecție foc/fum cu confirmare temporală.** Sistemul detectează prezența focului și a fumului în cadre. Pentru a reduce numărul de fals pozitive provocate de surse legitime (lumini, reflexii, obiecte de culoare apropiată), o alarmă de incendiu se generează doar după confirmarea pe o fereastră temporală configurabilă, nu pe baza unui singur cadru.
- F5 – Detecție arme.** Sistemul identifică obiectele de tip armă și marchează cadrele și camerele afectate.
- F6 – Clasificare acțiuni umane.** Sistemul clasifică acțiunile observate în cadrele video în categorii relevante pentru supraveghere (activitate normală, luptă sau vandalism).
- F7 – Alarmare.** La declanșarea unei detecții critice (persoană necunoscută, persoană necunoscută sau neautorizată într-o zonă restricționată, foc/fum confirmat, armă, acțiune periculoasă, plăcuță de înmatriculare neautorizată), sistemul generează o alarmă persistentă în baza de date, afișată în interfața utilizator cu severitatea și contextul corespunzător. Pentru a preveni inundarea operatorului cu alarme identice, este aplicată o *deduplicare temporală* pe perechea (cameră, tip), cu un cooldown fix de 30 de secunde. Alarmerile au un ciclu de viață propriu: pot fi vizualizate individual, marcate ca rezolvate (păstrând evidența utilizatorului care a făcut rezolvarea) și actualizate în masă (*bulk-update*).
- F7bis – Configurarea la runtime a detectoarelor.** Fiecare modul AI (recunoaștere facială, demografie, plăcuțe, foc, arme, acțiuni umane) poate

fi activat sau dezactivat individual fără repornirea backend-ului, prin endpoint-uri dedicate de tip `toggle`. Acest lucru permite operatorului să ajusteze sarcina GPU în funcție de scenariu (ex. dezactivarea citire plăcuțe pe camere de interior).

F8 – Gestiunea entităților (CRUD). Sistemul asigură operații complete (creare, citire, actualizare și ștergere) pentru entitățile principale: persoane (organizate pe departamente, cu fotografii multiple și embedding-uri faciale), zone (care includ indicatorul `is_restricted`), camere (gestionate prin descoperire dinamică sau prin salvare persistentă în baza de date), plăcuțe de înmatriculare și utilizatori (cu roluri specifice). Actualizările aplicate persoanelor sau plăcuțelor se reflectă imediat și consistent în indexul FAISS, dar și în listele de autorizare. În plus, sistemul pune la dispoziție endpoint-uri dedicate pentru interogarea *istoricului accesărilor* (asociat unei persoane sau plăcuțe), precum și pentru schimbarea parolei contului curent.

F9 – Jurnalizare și export. Detecțiile și alarmele sunt stocate în jurnale interogabile, prin intermediul tabelelor `detection_logs` și `alarms`. Aceste jurnale pot fi filtrate după interval de timp, cameră, tip de eveniment, status sau prin căutare după subiect. Pentru accesările persoanelor și ale vehiculelor există jurnale dedicate, `access_logs` și `vehicle_access_logs`. Jurnalurile de detecție pot fi exportate în format **CSV** pentru analiză externă, iar accesul vehiculelor este disponibil și prin endpoint-ul dedicat `/api/logs/vehicle`.

F10 – Statistici și panou de activitate. Sistemul agregă datele jurnalizate și pune la dispoziție rapoarte și grafice privind frecvența detecțiilor (`/api/logs/stats`, `/api/logs/timeseries`, `/api/logs/breakdown`), distribuția alarmelor pe camere și zone, demografia persoanelor identificate și alte metrice utile operatorilor. Pe pagina principală există un panou de *activitate recentă* actualizat în timp real prin WebSocket, care servește ca dashboard operațional.

Figura ?? sintetizează cele 10 cerințe funcționale sub forma unei diagrame use-case UML, plasând cei doi actori (Administrator, Operator) în raport cu funcționalitățile sistemului, grupate vizual pe domenii: detecție AI (F2–F6), interacțiune cu utilizatorul (F1, F7, F10) și administrare (F8, F9).

4.1.2 Cerințe non-funcționale

Cerințele nefuncționale fixează atributele de calitate sub care funcționalitățile enumerate anterior trebuie să opereze. Folosesc convenția consacrată în ingineria cerințelor, identificatori de tipul **NF x** (paralel cu **F x** de la §4.1.1), pentru a permite citarea lor din capitolele de proiectare, implementare și testare.

NF1 – Performanță (latență per cadru). Pipeline-ul principal de procesare a unui cadru, măsurat de la momentul achiziției până la difuzarea

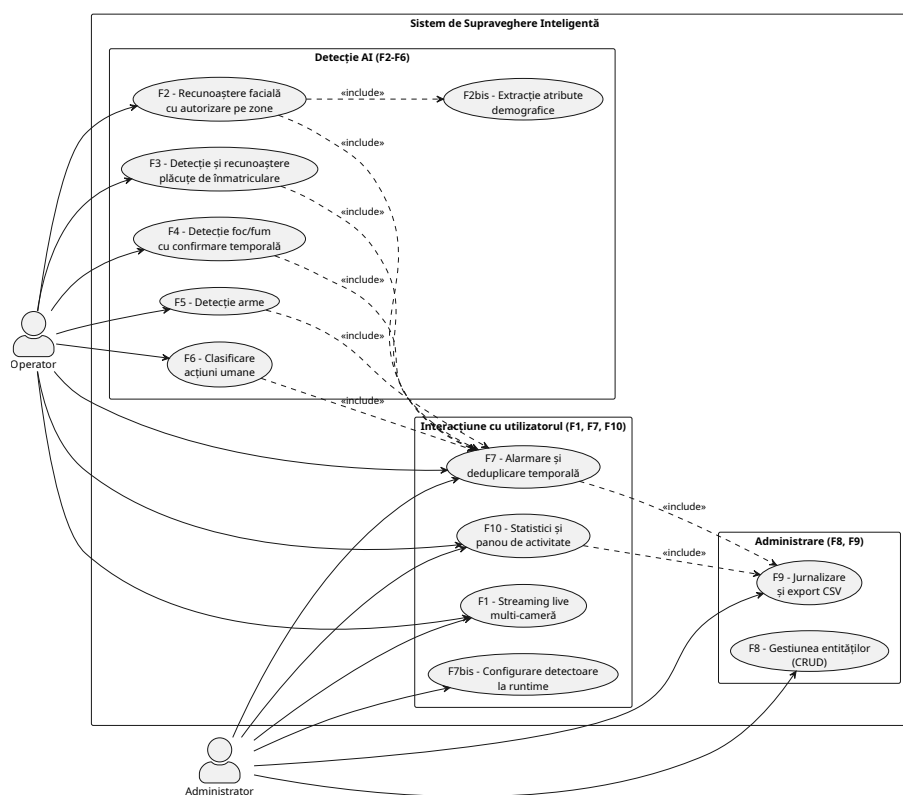


Figura 4.1: Diagrama use-case UML a sistemului de supraveghere inteligentă, cu actorii Administrator și Operator în raport cu cele 10 cerințe funcționale, grupate pe domenii (detectie AI, interacțiune cu utilizatorul, administrare).

rezultatelor către clienți, trebuie să se încadreze sub **100 ms per cadru**, ceea ce permite atingerea unei rate de aproximativ **30 FPS** pe fluxul live al camerei principale. Această țintă este derivată din modul de operare paralel al firelor de execuție: detectoarele costisitoare (SlowFast, YOLO) rulează în fire dedicate, astfel încât latența percepută de utilizator să fie dominată de cel mai rapid pipeline, nu de suma lor.

NF2 – Scalabilitate. Arhitectura trebuie să permită extinderea la **mai multe camere simultan** fără modificări structurale ale codului, ci doar prin replicarea firelor de captură și ajustarea cozilor de cadre. Modelele AI sunt încărcate o singură dată în memorie și partajate între fluxuri, pentru a evita duplicarea costurilor de inițializare GPU.

NF3 – Securitate. Operațiile sensibile ale API-ului (gestiunea persoanelor, a zonelor, a plăcutelor, a utilizatorilor și configurarea modulelor AI) sunt protejate prin **autentificare bazată pe JWT** și restricționate prin **control de acces bazat pe roluri (RBAC)** cu rolurile **admin** și **user**; operațiile administrative sunt impuse prin dependența **require_admin**. Parolele sunt stocate hash-uite (bcrypt), iar token-urile au timp de viață limitat (8 ore). Cheia de semnare JWT este citită din mediul de execuție (variabila **JWT_SECRET**); în lipsa ei, sistemul generează o cheie aleatorie per-proces și avertizează la pornire (token-urile devenind invalide la repornire). Canalul WebSocket de streaming este, la rândul lui, autentificat: token-ul JWT este transmis ca parametru de interogare (**/ws?token=...**), validat și re-verificat față de baza de date, astfel încât un cont șters nu mai poate continua să primească fluxul live. Endpoint-urile de control al camerelor (pornire/oprire, adăugare/eliminare cameră IP) necesită rolul de administrator. Comunicația cu baza de date PostgreSQL folosește credențiale dedicate, externalizate în mediul de execuție, separate de cele ale utilizatorilor finali.

NF4 – Fiabilitate. Indexul FAISS este reconstruit deterministic din PostgreSQL la fiecare pornire, astfel încât pierderea fișierului de index nu duce la pierderea datelor faciale. Pipeline-ul tolerează căderea temporară a unei camere fără a opri procesarea celorlalte fluxuri. Cozile de cadre folosesc politici de tipul *drop-oldest* la saturare, astfel încât un detector lent să nu blocheze fluxul principal.

NF5 – Configurabilitate la runtime. Administratorul poate activa/dezactiva fiecare modul AI fără restart al backend-ului și poate adăuga sau elimina dinamic camere IP. Pragurile de confidență ale detectoarelor și ferestrele de cooldown pentru alarme și jurnale sunt parametri centralizați în cod, modificabili fără reimplementare, dar nu sunt expuse spre ajustare în interfață.

NF6 – Mentenabilitate. Codul respectă o separare strictă pe module: fiecare detector (**facial_recognition_system**, **license_plate_recognition_system**, **fire_detection_system**, **weapon_detection_system**, **har_system**) este

o unitate de sine stătătoare, importată de `app.py` și plasată într-un fir de execuție dedicat. Adăugarea unui nou detector se reduce la implementarea aceleiași interfețe și conectarea unui nou fir cu cozile sale.

NF7 – Observabilitate. Toate detecțiile și alarmele sunt persistate cu timestamp, tip, cameră, subiect, confidență, severitate și metadata JSON, oferind un istoric complet și interogabil. Backend-ul raportează la pornire starea componentelor critice (GPU/CUDA, modele încărcate, conexiune DB).

NF8 – Uzabilitate. Interfața de utilizare este construită ca o aplicație web **responsive** (utilizabilă de la stația de operator până la dispozitive mobile pentru supraveghere de la distanță), organizată pe roluri: administratorul are acces la configurare și entități, operatorul are acces predominant la fluxurile live și la jurnalul de alarme.

NF9 – Portabilitate și mediu de execuție. Backend-ul este scris în Python (FastAPI), iar inferența AI necesită GPU compatibil CUDA pentru ca **InsightFace** și celelalte modele să atingă latențele țintă; sistemul detectează automat absența `CUDAExecutionProvider` și raportează acest lucru la pornire. Frontend-ul este o aplicație React standard, distribuibilă ca artefact static.

*[Diagramă: **tabel sintetic** (nu diagramă) care rezumă cerințele nefuncționale pe categorii (Performanță, Scalabilitate, Securitate, Fiabilitate, Uzabilitate) cu valoarea-țintă măsurabilă pentru fiecare. De realizat ca **tabular** în *LaTeX*, plasat la finalul subsecțiunii.]*

4.2 Cazuri de utilizare

După formularea cerințelor în Secțiunea 4.1, pasul firesc este transpunerea acestora în interacțiuni concrete între actori și sistem. Această secțiune identifică actorii, le delimitează responsabilitățile și descrie narativ cazurile principale de utilizare, oferind astfel puntea dintre *ce* oferă sistemul (cerințele **Fx**) și *cum* este folosit în practică.

Actori.

Sistemul distinge două roluri principale, mapate pe câmpul `role` al entității `user` din baza de date:

- **Administrator** – responsabil de configurarea sistemului. Are acces complet la gestiunea entităților (persoane, plăcuțe, zone, camere, utilizatori), la activarea/dezactivarea modulelor AI și activarea/dezactivarea camerelor de supraveghere. Tot ceea ce ține de *ce vede* sistemul și *pe cine îl recunoaște* este responsabilitatea acestui actor.
- **Utilizator obișnuit (Operator)** – responsabil de supravegherea operațională. Vizualizează fluxurile live, monitorizează panoul de alarme, le triază și le

marchează ca rezolvate și consultă jurnalele. Nu are dreptul să modifice configurația sistemului, să gestioneze entitățile sau să creeze conturi noi.

Ambii actori partajează un nucleu comun de cazuri de utilizare (autentificare, schimbare parolă, vizualizare live, consultare jurnale), peste care se adaugă privilegiile specifice rolului. Sistemul mai interacționează implicit cu *actori externi* pasivi – camerele IP, modelele AI încărcate și baza de date PostgreSQL – dar aceștia nu sunt utilizatori în sensul UML și sunt tratați ca sisteme suport.

[Diagramă: **diagrama de cazuri de utilizare UML cu cei doi actori** (Administrator, Operator) plasați la margini, conectați prin linii la elipsele cazurilor de utilizare descrise mai jos. Cazurile UC1–UC3, UC10, UC11 și consultarea jurnalelor din UC12 sunt partajate (linie de la ambii actori). UC4–UC9, precum și vizualizarea statisticilor din UC12, sunt accesibile doar Administratorului. Folosiți relații «include» pentru autentificare (UC1) către toate celelalte cazuri și «extend» pentru rezolvarea alarmei (UC11) către monitorizarea alarmelor (UC10).]

Cazuri de utilizare.

Cazurile sunt numerotate UC1...UC12 și descrise în formă narativă (pre-condiție, scenariu principal, post-condiție), cu trimitere la cerințele funcționale pe care le acoperă.

UC1 – Autentificare în sistem. *Actori:* Administrator, Operator. *Pre-condiție:* utilizatorul are un cont creat. *Scenariu:* utilizatorul introduce numele de utilizator și parola în pagina de login; backend-ul verifică hash-ul parolei, generează un token JWT semnat și îl returnează clientului, care îl stochează local și îl atașează tuturor cererilor ulterioare. *Post-condiție:* sesiunea este activă, iar UI-ul afișează doar componentele permise rolului utilizatorului. *Acoperă:* parte din NF3 (autentificare JWT).

UC2 – Vizualizare fluxuri live multi-cameră. *Actori:* Administrator, Operator. *Pre-condiție:* cel puțin o cameră este pornită. *Scenariu:* utilizatorul deschide pagina principală; clientul stabilește o conexiune WebSocket cu backend-ul; backend-ul difuzează cadrele procesate (cu detecțiile suprapuse) către toți clienții conectați, la o rată țintă de aproximativ 30 FPS. Utilizatorul poate selecta camera afișată în prim-plan dintr-un grid. *Post-condiție:* fluxurile sunt afișate în timp real cu adnotările vizuale ale detectoarelor active. *Acoperă:* F1 (și indirect F2–F6 pentru adnotările suprapuse).

UC3 – Schimbare parolă cont propriu. *Actori:* Administrator, Operator. *Scenariu:* utilizatorul autentificat introduce noua parolă; backend-ul o validează, o hash-uieste și o salvează în baza de date prin endpoint-ul `/api/users/me/password`. *Post-condiție:* la următoarea autentificare se folosește noua parolă. *Acoperă:* parte din NF3.

UC4 – Înregistrare persoană nouă. *Actor:* Administrator. *Scenariu:* dintr-un singur formular, administratorul completează metadatele persoanei

(nume, `employee_id`, departament, zone autorizate) și selectează una sau mai multe imagini faciale. La trimiterea formularului, interfața orchestrează automat două apeluri REST succesive: mai întâi `/api/persons/register`, care persistă metadatele și returnează `person_id`-ul nou creat; apoi `/api/persons/{person_id}/faces`, unde backend-ul extrage pentru fiecare imagine embedding-ul InsightFace 512-d, îl salvează în tabela `face_embeddings` și reconstruiește indexul FAISS din baza de date. Din perspectiva administratorului fluxul este un singur pas, fără a mai fi nevoie de un script extern pentru capturarea fețelor. *Excepție:* o imagine în care nu se detectează *exact* o față (zero sau mai multe fețe, deci embedding `None`) este ignorată și raportată în lista de erori, fără a întrerupe procesarea celorlalte. *Post-condiție:* după încărcarea imaginilor, persoana devine identificabilă în fluxurile live. *Acoperă:* F2, F8.

UC5 – Gestiune zone. *Actor:* Administrator. *Scenariu:* administratorul creează, modifică sau șterge o zonă, definind numele, descrierea și indicatorul `is_restricted`. Autorizarea propriu-zisă nu se setează pe zonă, ci pe persoană – prin lista `authorized_zones` a fiecărei persoane (UC4) – zona fiind referită prin nume. Schimbările se reflectă imediat în logica de verificare a accesului, după reîmprospătarea cache-urilor de zone. *Post-condiție:* prezența unei persoane necunoscute sau neautorizate într-o zonă restricționată va genera o alarmă (vezi UC10). *Acoperă:* F2, F8 și parte din F7.

UC6 – Gestiune plăcuțe. *Actor:* Administrator. *Scenariu:* administratorul înregistrează o plăcuță (număr, tip de vehicul, nume și ID proprietar, indicator `is_authorized`, dată de expirare, note), o modifică sau o șterge. Spre deosebire de persoane, plăcuțele nu au autorizare pe zone – o plăcuță este considerată autorizată dacă numărul ei este regăsit în baza de date la momentul citirii OCR. *Post-condiție:* citirile de plăcuțe sunt comparate cu baza înregistrată și *jurnalizate* cu status `authorized/unauthorized`; o plăcuță neautorizată cu citire OCR validă generează în plus o alarmă de severitate `high` (vezi UC10 și F7). *Acoperă:* F3, F7, F8, F9.

UC7 – Adăugare/eliminare cameră. *Actor:* Administrator. *Scenariu:* administratorul adaugă o cameră IP indicând URL-ul fluxului, un identificator unic și locația, sau adaugă o cameră persistentă din baza de date `cameras-db`; backend-ul deschide fluxul, pornește firul de captură și înregistrează camera în lista de camere active. Operația simetrică de eliminare oprește firul și eliberează resursa. *Post-condiție:* camera apare/dispare din grid-ul live. *Acoperă:* F1, F8.

UC8 – Configurarea modulelor AI la runtime. *Actor:* Administrator. *Scenariu:* administratorul accesează panoul *Intelligence Settings* și activează/dezactivează individual recunoașterea facială, demografia, plăcuțele, focul, armele și acțiunile umane prin endpoint-urile `toggle`. Modificarea are efect imediat – firul de execuție corespunzător începe sau încetează să

mai pună cadre în coadă – fără repornirea backend-ului. *Post-condiție:* pipeline-ul rulează cu noul set de detectoare active. *Acoperă:* F7bis, NF5.

UC9 – Gestiune utilizatori. *Actor:* Administrator. *Scenariu:* administratorul creează un cont nou specificând nume, parolă inițială și rol; poate ulterior să modifice rolul sau să șteargă conturi (cu interdicția de a-și șterge propriul cont). *Post-condiție:* setul de utilizatori autorizați să acceseze sistemul este actualizat. *Acoperă:* F8, NF3.

UC10 – Monitorizare alarme. *Actori:* Administrator, Operator. *Pre-condiție:* pipeline-ul de procesare rulează și are detectoare active. *Scenariu:* ori de câte ori se declanșează o detecție critică (persoană necunoscută, persoană necunoscută/neaautorizată într-o zonă restricționată, foc/fum, armă, acțiune periculoasă, plăcuță de înmatriculare neautorizată), backend-ul, după aplicarea cooldown-ului de deduplicare, salvează o alarmă în baza de date împreună cu o captură (snapshot). Clientul interoghează periodic endpoint-ul `/api/alarms` și afișează alarmele în panoul de alarme, cu severitatea, camera, timestamp-ul și captura asociată. *Post-condiție:* alarma este vizibilă tuturor operatorilor. *Acoperă:* F7, F2–F6, NF1.

UC11 – Rezolvare alarmă. *Actor:* Operator (sau Administrator). *Pre-condiție:* există cel puțin o alarmă activă. *Scenariu:* operatorul selectează o alarmă din panou și o marchează ca rezolvată; backend-ul persistă identitatea celui care a rezolvat alarma și timestamp-ul rezolvării. Operația poate fi aplicată în masă (*bulk-update*) pe mai multe alarme. *Extinde:* UC10. *Post-condiție:* alarma trece în starea **resolved** și dispare din panoul de alarme active. *Acoperă:* F7.

UC12 – Consultare jurnale și statistici. *Scenariu:* Consultarea jurnalelor (*actori:* Administrator, Operator) – utilizatorul deschide pagina de jurnale și aplică filtre pe interval de timp, cameră, tip de eveniment, status și căutare după subiect; backend-ul returnează rezultatele paginate, care pot fi exportate în format CSV. *Vizualizarea statisticilor* (*actor:* doar Administrator) – pagina de statistici afișează grafice agregate (frecvența detecțiilor în timp, breakdown pe tipuri, distribuția pe camere). *Post-condiție:* utilizatorul are o vedere asupra activității sistemului pe perioada selectată. *Acoperă:* F9, F10.

Tabelul de trasabilitate cerință→caz de utilizare se regăsește implicit prin trimerile *Acoperă:* din fiecare descriere de mai sus și va fi folosit ca referință în Capitolul 7, unde fiecare caz de utilizare este verificat printr-un scenariu de test corespunzător.

4.3 Arhitectura generală a sistemului

După stabilirea cerințelor (§4.1) și a cazurilor de utilizare (§4.2), această secțiune prezintă arhitectura sistemului la nivel macro, evidențiind structura fundamen-

tală, descompunerea în componente logice și distribuția acestora pe noduri fizice. Detaliile interne ale pipeline-ului multi-threaded și ale interfețelor REST/WebSocket sunt amânate pentru Secțiunile 4.4–4.6.

Sistemul este construit ca o aplicație web cu **trei straturi logice**: prezentare (clientul React), aplicație (backend-ul FastAPI cu pipeline-ul AI integrat) și persistență (baza de date PostgreSQL plus indexul FAISS reconstruit la pornire). Aceste straturi sunt utilizate de **cinci tipuri de actori tehnici**: utilizatorii umani prin browser, camerele video ca surse externe de date, modelele AI încărcate în memorie ca dependente în proces, baza de date ca sistem suport și GPU-ul ca resursă de calcul. Alegerea unei arhitecturi monolitice pentru backend, în care toate detectoarele rulează în același proces și partajează memoria GPU, este justificată în §4.4.3.

[Diagramă: **diagramă bloc de ansamblu** care arată cele trei straturi pe orizontală (Prezentare | Aplicație | Persistență), cu săgeți de comunicare etichetate pe protocol: HTTPS/REST și WebSocket între client și backend, RTSP/HTTP/MJPEG între camere și backend, SQL prin driver *psycopg2* între backend și PostgreSQL, apeluri în proces între backend și modelele AI încărcate pe GPU. Plasată la începutul Secțiunii 4.3, înainte de §4.3.1.]

4.3.1 Modelul client-server

Sistemul urmează un model client-server clasic, cu o singură instanță a serverului care deservește simultan mai mulți clienți web. Conexiunile sunt asimetrice: pentru operațiile tranzacționale (login, CRUD pe entități, interogări de jurnale, preluarea alarmelor) se folosește un canal **REST sincron** peste HTTP/HTTPS, iar pentru transmiterea continuă a cadrelor video procesate se folosește un canal **WebSocket** peste TCP, în care fluxul de date este predominant unidirecțional (server→client).

4.3.1.1 Clientul

Frontend-ul este o aplicație **React 19** compilată cu Create React App și stilizată cu Tailwind, livrabilă ca artefact static. Este o aplicație de tip *single-page*, organizată pe ecrane comutate printr-o stare internă (**activeTab**), fără un router dedicat: **CameraGrid**, **PersonManagement**, **PlateManagement**, **ZoneManagement**, **CameraManagement**, **UserManagement**, **AlarmManagement**, **Logs**, **Statistics**, **IntelligenceSettings**, **RecentActivity**. Clientul nu execută niciun cod AI – rolul său este de a afișa cadrele primite prin WebSocket, de a colecta input-ul utilizatorului și de a-l transmite ca apeluri REST.

4.3.1.2 Serverul

Backend-ul este o aplicație **FastAPI** pornită pe portul 8000 din `app.py`, încărcat ca o singură instanță Uvicorn (fără reload, pentru a evita dubla inițializare a modelelor pe GPU). Endpoint-urile sunt protejate prin token JWT și verificări de rol (dependențele `require_admin` / `get_current_user`), aplicate înainte de

invocarea handler-ului: operațiile administrative (controlul camerelor, gestiunea entităților, comutatoarele AI) necesită rolul de administrator, iar endpoint-urile operaționale (starea sistemului, listarea camerelor, jurnale) necesită doar autentificare. Canalul WebSocket este, la rândul lui, autentificat prin token JWT transmis ca parametru de interogare – aspect detaliat în §4.8. Pe lângă de-servirea cererilor HTTP, serverul găzduiește pipeline-ul AI ca un set de fire de execuție de fundal, descrise în §4.4.

4.3.1.3 Sursele video

Camerele sunt actori externi care nu inițiază niciodată conexiuni către server; serverul este cel care le deschide fluxurile prin OpenCV – camera locală a stației (webcam USB, `cv2.VideoCapture(0)`) și camere IP adăugate prin URL de flux (RTSP/HTTP/MJPEG). Eșecul unei camere – timeout, deconectare – nu propagă o eroare în lanț, ci doar marchează camera ca inactivă și permite repornirea ulterioară.

4.3.1.4 Persistența

Stratul de persistență combină **PostgreSQL** utilizat pentru stocarea permanentă cu un **index FAISS** menținut în memoria backend-ului pentru căutarea vectorială rapidă a embedding-urilor faciale 512-d. Indexul nu reprezintă mecanismul principal de stocare a datelor, ci este reconstruit determinist din PostgreSQL la fiecare pornire prin modulul `faiss_index.py`, astfel încât pierderea fișierului de cache nu afectează integritatea informațiilor.

4.3.2 Diagrama de componente

La nivel intern, backend-ul este descompus în următoarele componente logice. Fiecare componentă este un modul Python sau un grup de fire de execuție cu responsabilitate clară, cuplate prin cozi `queue.Queue` sau prin apeluri directe.

Modul de captură. Implementat în `app.py` sub forma unui fir de execuție per cameră activă, care deschide fluxul prin OpenCV, citește cadrele în mod sincron și le distribuie – prin clonare `frame.copy()` – în cozile de procesare ale fiecărui detector activ și în coada `raw_frame_queue` folosită ca fallback pentru fuziune.

Modul AI. Containerul pentru toate detectoarele specializate, încărcate o singură dată la pornire și partajate între fluxurile camerelor. Conține următoarele sub-module independente, fiecare cu propriul fir de execuție și propria pereche de cozi (`*_processing_queue` de intrare, `*_results_queue` de ieșire):

- **facial_recognition_system** – detecție de fețe (**InsightFace**), embedding-uri 512-d, căutare în FAISS, extragere atribute demografice (vârstă, gen prin InsightFace; emoție prin Mini-Xception/FER).

- `license_plate_recognition_system` – detecție și OCR pe plăcuțe (**YOLOv8** pentru localizare + **EasyOCR** pentru text).
- `fire_detection_system` – detector foc/fum cu **confirmare temporală** pe fereastră multi-cadru, pentru reducerea fals pozitivelor.
- `weapon_detection_system` – detector **YOLOv8m** fine-tuned pe clasa unică `weapon`, antrenat pe Zenodo *Dangerous Items* + Roboflow *SOHAS*.
- `har_system` – recunoaștere acțiuni umane folosind **SlowFast-R50** (clipuri de 64 cadre, două căi slow/fast), care clasifică fiecare clip în acțiunile `normal`, `fight` și `vandalism`.

Modul de fuziune. Firul *result merger* colectează rezultatele parțiale din toate cozile de ieșire ale detectoarelor active, le asociază pe baza câmpului `frame_id` și produce un cadru final adnotat (cu bounding box-uri, etichete, scoruri de confidență), plus un set agregat de detecții. Tot aici se aplică **logica de declanșare a alarmelor** (verificarea zonelor restricționate, deduplicarea pe cooldown) și **politica de jurnalizare** (un singur log per cameră, tip, subiect) într-o fereastră scurtă).

Modul de control al accesului. Cache-urile `_person_zones_cache` și `_camera_zone_cache` și logica asociată din `app.py` verifică, pentru fiecare *persoană* identificată, dacă aceasta are dreptul de acces în zona asociată camerei (comparând zona camerei cu lista `authorized_zones` a persoanei). Cache-urile sunt reîmprospătate periodic (TTL) și forțat la modificarea entităților relevante. Plăcuțele nu participă la acest mecanism, autorizarea lor fiind globală (§4.1.1).

API REST. Stratul de controlere FastAPI definit în `app.py`, organizat pe domenii: `/api/login`, `/api/persons`, `/api/plates`, `/api/zones`, `/api/cameras-db`, `/api/users`, `/api/logs`, `/api/alarms`, `/api/statistics`, plus endpoint-urile `toggle` pentru fiecare modul AI. Fiecare controler validează token-ul JWT, aplică verificarea de rol și delegă efectiv către modulele de business sau către `database_manager.py`.

Gestiunea WebSocket. Endpoint-ul `/ws` autentifică clientul (token JWT ca parametru de interogare) și îi alocă o **coadă proprie** (`queue.Queue`), înregistrată în dicționarul `client_queues`; modulul de fuziune nu publică într-o coadă partajată, ci difuzează fiecare cadru procesat către coada fiecărui client prin metoda `_broadcast_frame` (politică *drop-oldest* per client, astfel încât un client lent să-și piardă doar propriile cadre vechi). Mesajele difuzate (câmpul `type: video_frame`) conțin cadrul codificat (JPEG la calitate 75, redimensionat la maxim 800 px lățime, Base64), rezultatele detectoarelor pe câmpuri separate (`face_results`, `plate_results`, etc.) și starea modulelor (flag-urile `*_enabled`). Comunicarea este *strict unidirecțională* (server→client): handler-ul doar trimite cadre, fără a citi mesaje de la client. Alarmerile nu sunt transmise prin acest canal, ci preluate separat prin REST (§4.6).

Layer de persistență. Modulul `database_manager.py` centralizează toate accesele la PostgreSQL printr-o conexiune `psycopg2` persistentă (cu `RealDictCursor`), oferind operații CRUD și interogări parametrizate pentru toate entitățile. Indexul FAISS, deși tehnic în memoria procesului, este considerat parte a acestui layer pentru că este reconstruit din DB. Datele binare sunt păstrate tot în baza de date: embedding-urile faciale ca BYTEA în `face_embeddings`, iar capturile de alarmă (snapshot-urile) ca șiruri Base64 în coloana `snapshot` a tabelului `alarms`; sistemul nu expune un serviciu de fișiere statice.

Cuplajul între componente este **slab**: fiecare detector poate fi activat/dezactivat independent (vezi UC8) fără a afecta restul, iar adăugarea unui nou detector se reduce la implementarea aceleiași interfețe (*intrare*: cadru, *ieșire*: listă de detecții) și conectarea unui nou fir cu cozile sale.

[Diagramă: **diagrama de componente UML** cu Modul Captură pe stânga, Modul AI ca component *compus* cu cele cinci sub-module *nested* înăuntru, Modul Fuziune ca hub central, iar pe dreapta API REST + Gestiune WebSocket conectate la Layer Persistență (PostgreSQL + FAISS). Folosiți interfețe *lollipop* (cercuri) pentru a marca contractele între componente, în special cele oferite de Modul AI către Modul Fuziune (interfața Detector). Plasată în §4.3.2.]

4.3.3 Diagrama de implementare (deployment)

Distribuția fizică a sistemului este simplă în varianta de bază – un singur nod server pe care rulează tot ce ține de procesare – și se poate extinde modular dacă numărul de camere sau de clienți crește. Nodurile descrise mai jos sunt suficiente pentru scenariul demonstrativ al lucrării și pentru un deployment pilot.

4.3.3.1 Nodul client (browser)

Orice mașină cu un browser modern (Chrome, Firefox, Edge) care suportă WebSocket și ES2020. Nu există dependente instalate local: aplicația React este servită ca artefact static și se conectează la backend prin URL-ul configurat în `frontend/.env`. În scenariul de operare pot exista mai mulți clienți simultan (un dispecer pe stație fixă, plus operatori pe dispozitive mobile).

4.3.3.2 Nodul server cu GPU

Mașina principală a sistemului, pe care rulează:

- procesul Python `app.py` (FastAPI + Uvicorn) cu toate firele AI;
- modelele AI încărcate în memoria GPU (InsightFace, YOLOv8, SlowFast, modelul de foc); modelul de emoție FER (Mini-Xception, TensorFlow) este ținut deliberat pe CPU, pentru a evita conflictele de runtime cu `onnxruntime-gpu`;
- indexul FAISS în memoria RAM;

- runtime-ul `onnxruntime-gpu` cu `CUDAExecutionProvider`, validat la pornire prin `start_enhanced_system.py`.

Cerințele minime hardware sunt impuse de modelele AI: un GPU cu suport CUDA și cel puțin 8 GB VRAM pentru încărcarea simultană a tuturor detectoarelor; un CPU multi-core pentru cele ~ 7 fire de execuție; un SSD pentru log-uri și capturi.

4.3.3.3 Nodul bază de date

PostgreSQL poate coexista cu backend-ul pe același host (varianta implicită – conexiune prin `localhost`) sau poate fi externalizat pe un nod separat pentru scenarii de producție, accesat prin TCP cu credențiale dedicate. Externalizarea aduce avantaje de backup și replicare, dar nu este obligatorie pentru lucrare.

4.3.3.4 Camere IP

Camerele sunt noduri independente, conectate la aceeași rețea locală cu serverul, care expun fluxuri RTSP, MJPEG sau HTTP. Identificarea lor este făcută prin URL plus un identificator logic (`CAM-01`, `CAM-02` etc.). Latența rețelei dintre camere și server contribuie direct la bugetul de 100 ms al cerinței NF1, motiv pentru care se recomandă rețea cablată sau Wi-Fi dedicat.

4.3.3.5 Storage (optional)

Pentru deployment-urile cu volum mare de capturi (clipuri video, imagini de alarmă) se poate adăuga un nod de storage dedicat – NAS sau bucket S3-compatibil – montat de server ca volum sau accesat prin client-ul corespunzător. În varianta de bază a lucrării, fișierele sunt stocate pe disk-ul local al server-ului, în directorul `Facial_detection/`.

[Diagramă: **diagrama UML de deployment** cu patru noduri principale desenate ca cuburi 3D: Client (Browser), Server GPU (cu artefactele `app.py`, modelele AI, indexul FAISS desenate ca rectangle nested), PostgreSQL (poate fi pe același nod sau separat – arătați ambele variante printr-un dashed-box opțional), Camere IP (multiple instanțe, indicate prin notația $\{1..n\}$). Etichete pe legături: `HTTPS/WebSocket client↔server`, `RTSP/HTTP server↔camere`, `TCP/SQL server↔DB`. Opțional: un al cincilea nod Storage legat de server prin `NFS/S3`. Plasată în §4.3.3.]

4.4 Proiectarea pipeline-ului multi-threaded

Pipeline-ul multi-threaded este decizia arhitecturală centrală a sistemului – modul în care backend-ul reușește să ruleze simultan cinci modele AI eterogene (recunoaștere facială, plăcuțe, foc/fum, acțiuni umane, arme) peste fluxul aceleiași camere și să livreze rezultatele agregate în limita de latență NF1.

Această secțiune descrie structura firelor de execuție, mecanismele de sincronizare alese și motivează de ce o execuție paralelă în-proces este preferabilă unei secvențiale sau unei descompunerii în microservicii.

Decizia de bază este: **un fir de execuție per responsabilitate, cozi thread-safe cu capacitate limitată ca mecanism de cuplare slabă, un singur proces care găzduiește toate modelele pe GPU**. Această alegere derivă din trei constrângeri: (i) modelele AI au timpi de inferență neomogeni (de la sub 10 ms pentru YOLO pe persoane până la peste 100 ms pentru SlowFast pe un clip de 64 cadre), (ii) inițializarea GPU a fiecărui model este costisitoare și trebuie făcută o singură dată, (iii) operatorul trebuie să poată activa/dezactiva dinamic detectoarele (cerința F7bis, NF5).

[Diagramă: **diagrama de flux a pipeline-ului** cu cele 7 fire reprezentate ca dreptunghiuri verticale, cozile `queue.Queue` ca cilindri orizontali între ele, și fluxul cadrelor mergând de la stânga (`CaptureThread`) prin cele cinci fire AI paralele către dreapta (`MergingThread` → `_broadcast_frame` → cozile per-client `client_queues` → clienții `WebSocket`). Marcați pe fiecare fir timpul tipic de inferență și pe fiecare coadă `maxsize=10` cu politica de saturare (omitere la intrare, drop-oldest la ieșire). Plasată la începutul §4.4.1.]

4.4.1 Cele 7 fire de execuție

Backend-ul pornește, la initializarea instanței `EnhancedSurveillanceSystem`, exact șapte fire de execuție daemon (vizibile în `app.py` prin numele `CaptureThread`, `FaceProcessingThread`, `PlateProcessingThread`, `FireProcessingThread`, `HARProcessingThread`, `WeaponProcessingThread`, `MergingThread`). Fiecare fir are o singură responsabilitate și interacționează cu restul *exclusiv* prin cozile partajate, evitând astfel sincronizarea prin variabile partajate la nivel fin.

4.4.1.1 Firul 1 – Captura video

Firul `CaptureThread` citește ciclic câte un cadru de la fiecare cameră activă – camera locală a stației, tratată separat ca CAM-01 prin `self.video_capture`, și camerele IP înregistrate în structura `self.cameras` (CAM-02, CAM-03, ...) – prin `cv2.VideoCapture.read()`, cu reconectare automată a camerelor IP căzute. Fiecare cadru primește un `frame_id` monoton crescător și, sub rezerva unui mecanism de *frame-skip* (se procesează un cadru la fiecare `frame_skip`), este transmis metodei `_dispatch_frame`. Aceasta depune, pentru fiecare detector activ, o copie a cadrului (`frame.copy()`) în coada de procesare corespunzătoare (`face_processing_queue`, `plate_processing_queue`, `fire_processing_queue`, `har_processing_queue`, `weapon_processing_queue`) și, doar dacă toate cozile detectoarelor active au acceptat cadrul, o copie în `raw_frame_queue`, folosită de fuziune ca *fallback* pentru cadrele pentru care nu au sosit încă rezultatele detectoarelor.

4.4.1.2 Firul 2 – Recunoaștere facială și analiză demografică

Firul `FaceProcessingThread` consumă din `face_processing_queue`, rulează detecția fețelor cu `InsightFace`, calculează embedding-urile 512-d, le caută în indexul FAISS în memoria procesului și asociază fiecărei detecții vârsta, genul (din `InsightFace`) și emoția (din modelul `Mini-Xception/FER`). Toate aceste operații sunt încapsulate într-un singur fir pentru că au aceeași intrare (regiunea facială) și aceeași localitate de cache GPU; separarea recunoașterii faciale de demografie ar dubla detecția fețelor fără beneficiu real.

4.4.1.3 Firul 3 – Plăcuțe de înmatriculare

Firul `PlateProcessingThread` rulează detecția plăcuțelor cu `YOLOv8` specializat, urmată de OCR-ul `EasyOCR` pe fiecare regiune detectată. Combinația detector+OCR este menținută în același fir deoarece OCR-ul operează doar pe ROI-urile produse de detector, iar separarea ar introduce o coadă inutilă pentru ROI-uri.

4.4.1.4 Firul 4 – Foc și fum

Firul `FireProcessingThread` rulează modelul de detecție foc/fum și, important, menține **starea temporală** necesară confirmării pe fereastră multi-cadru (cerința F4). O detecție singulară nu produce alarmă – doar o detecție confirmată pe câteva cadre consecutive declanșează câmpul `alert` pe rezultatul publicat. Această stare este intern firului și nu necesită sincronizare.

4.4.1.5 Firul 5 – Recunoaștere acțiuni umane (HAR)

Firul `HARProcessingThread` este cel mai costisitor: acumulează cadre într-un buffer intern și, când are clipul complet de 64 cadre la `clip_duration_sec=2.0`, rulează `SlowFast-R50` cu pathway-urile `slow` (8 cadre) și `fast` (32 cadre) la `crop_size=224`, clasificând clipul în `normal`, `fight` sau `vandalism`. Datorită costului ridicat și a inerent batched-ness-ului `SlowFast`-ului, firul tinde să fie cel care impune ritmul throughput-ului HAR – de aceea este izolat și nu blochează celelalte detectoare.

4.4.1.6 Firul 6 – Arme

Firul `WeaponProcessingThread` rulează modelul `YOLOv8m` fine-tuned pe clasa unică `weapon`. Este separat de detectorul de plăcuțe (deși ambele folosesc `YOLO`) pentru că instanțele de model sunt distincte (greutăți diferite, clase diferite) și pentru că severitatea detecției de armă justifică un fir dedicat care nu poate fi blocat de OCR-ul plăcuțelor.

4.4.1.7 Firul 7 – Fuziune

Firul `MergingThread` este nucleul de agregare: consumă neblokant din toate cozile de rezultate (`face_results_queue`, `plate_results_queue`, `fire_results_queue`,

`har_results_queue`, `weapon_results_queue`) plus din `raw_frame_queue`, asociază rezultatele după `frame_id` și produce mesajul final pentru clienți (cadru adnotat + listă de detecții + alarme noi). Tot aici se aplică logica de declanșare a alarmelor (verificarea zonelor restricționate via `_person_zones_cache`, deduplicarea pe baza dicționarului `alarm_cooldowns` cu un cooldown de 30 s, jurnalizarea cu deduplicare pe `log_cooldowns` de 5 s). Mesajul final este difuzat prin metoda `_broadcast_frame`, care îl depune în coada proprie a fiecărui client WebSocket conectat (dicționarul `client_queues`), fără a trece printr-o coadă de ieșire partajată.

4.4.1.8 Justificarea separării

Alocarea pe fire respectă două principii: **omogenitatea computațională** (operații care partajează intrarea și cache-ul GPU stau împreună – ex. recunoaștere facială cu demografie, detecție plăcuță cu OCR) și **izolarea costului** (firele scumpe – HAR, SlowFast – sunt singure pentru a nu bloca celelalte). Numărul 7 nu este un obiectiv în sine, ci o consecință a celor cinci familii de detecții cerute plus capătul de pipeline (captură) și nucleul de agregare (fuziune).

4.4.2 Mecanisme de sincronizare

Sincronizarea este realizată prin **cozi thread-safe cu capacitate limitată** ca mecanism principal și prin **lock-uri fine de tip threading**. Lock pentru structurile de stare partajată (cache de zone, dicționare de cooldown, lista de camere). Această împărțire reduce semnificativ riscul de *deadlock* și de *race condition*, pentru că majoritatea fluxului de date trece prin cozi, unde sincronizarea este implementată odată în standard library.

4.4.2.1 Cozi thread-safe

Toate cele unsprezece cozi folosite de pipeline sunt `queue.Queue(maxsize=10)` – cozi FIFO cu blocaj intern și suport nativ pentru `block=False` și `timeout=...`. Capacitatea limitată la 10 este o decizie explicită: un buffer scurt înseamnă latență mică (cadrul cel mai vechi în pipeline este de cel mult 10 cadre vechi), dar înseamnă și că saturarea trebuie tratată. Politica de saturare diferă în funcție de direcția cozii, dar urmărește același scop – a nu lăsa coada să crească: la **intrare** (cozile de procesare `*_processing_queue` și `raw_frame_queue`, alimentate de `CaptureThread` prin `_dispatch_frame`), când coada unui detector este plină, cadrul nou este pur și simplu *omis* (incrementând un contor de `queue_skips`), pentru că detectorul respectiv încă procesează un backlog și nu are rost să-l adâncim; la **ieșire** (cozile `*_results_queue` și cozile per-client WebSocket), producătorul aplică **drop-oldest** – scoate explicit elementul cel mai vechi cu `get(block=False)` și inserează noul rezultat. În ambele cazuri principiul este același: *contează cel mai recent cadru/rezultat, nu acumularea unei coade întregi de date vechi*.

4.4.2.2 Partajarea cadrului curent

Cadrela citite de `CaptureThread` sunt *clonate* (`frame.copy()`) înainte de a fi distribuite în cozile detectoarelor. Această decizie – aparent risipitoare – elimină complet sincronizarea pe conținutul cadrului: fiecare fir AI lucrează pe propria copie, fără riscul ca alt fir să modifice pixelii sub picioarele lui. Costul memoriei este absorbit de capacitatea limitată a cozilor (cel mult $11 \times 10 = 110$ cadre simultan în RAM, în jur de 200 MB pentru rezoluții HD), iar costul copierii este sub 1 ms, neglijabil față de bugetul NF1.

4.4.2.3 Mecanismul *ultimul cadru disponibil*

Firul de fuziune nu așteaptă să aibă rezultatele *tuturor* detectoarelor pentru fiecare `frame_id`. Așteptarea ar reduce throughput-ul la minimum dintre detectoare (HAR, care livrează rezultate o dată la ~ 2 s, ar limita restul la 0.5 FPS). În schimb, fuziunea aplică o politică de tip *ultimul cadru disponibil*: consumă neblockant din toate cozile de rezultate, păstrează în structuri locale ultimele rezultate per cameră primite de la fiecare detector și le suprapune pe cadrul cel mai recent. Astfel, detecțiile rapide (fețe, plăcuțe) sunt afișate la rata camerei, iar detecțiile lente (HAR) sunt suprapuse cu valorile lor cele mai recente până când vine o actualizare. Acest mecanism este cel care permite respectarea cerinței NF1 de 30 FPS *indiferent* de detectorul cel mai lent activ.

4.4.2.4 Stare partajată protejată prin lock-uri

Câteva structuri trebuie totuși accesate concurent: lista camerelor active (`cameras_lock`), dicționarele de cooldown pentru alarme și jurnale (`alarm_lock`, `log_lock`), cache-urile de autorizare pe zone (`_zone_cache_lock`) și dicționarul cozilor per-client WebSocket (`client_lock`). Pentru toate acestea se folosesc lock-uri de tip `threading.Lock`, cu scope foarte restrâns (câteva linii înăuntrul blocului `with self.X_lock:`), pentru a minimiza timpul în care un fir este blocat.

4.4.3 Justificarea procesării paralele

Alternativa naturală la pipeline-ul paralel este **procesarea secvențială** – pentru fiecare cadru, se aplică pe rând detector 1, detector 2, ..., detector 5, apoi se trimite rezultatul la client. Această variantă a fost respinsă din mai multe motive concrete, măsurabile pe propriul cod.

4.4.3.1 Utilizarea GPU-ului

Modelele AI partajează aceeași memorie GPU, dar nu același tip de calcul: InsightFace face inferență single-shot pe regiuni mici, YOLO face inferență pe imagine întreagă, SlowFast face inferență pe clip 3D temporal. Rularea lor în fire separate permite suprapunerea **copierii host→device** a unui detector cu **kernel execution** a altuia, prin streams CUDA, ceea ce într-o execuție secvențială ar rămâne timp mort. În plus, ONNX Runtime și PyTorch folosesc

thread pool-uri interne care exploatează concurența la nivel de operator – iar acestea sunt active doar dacă apelurile vin din fire diferite.

4.4.3.2 Latența percepută per modul

Într-un pipeline secvențial, latența percepută de utilizator pentru *orice* detector este suma timpilor de inferență ai *tuturor* detectoarelor active. Dacă SlowFast costă ~120 ms, recunoașterea facială ~20 ms și plăcuțele ~40 ms, atunci chiar și un client interesat doar de fețe ar primi rezultate la ~180 ms după captură, depășind clar pragul NF1 de 100 ms. În pipeline-ul paralel, fiecare modul își are propria latență individuală, iar mecanismul *ultimul cadru disponibil* face ca afișarea să fie limitată de cel mai rapid detector, nu de cel mai lent.

4.4.3.3 Dezactivarea selectivă

Cerința F7bis cere posibilitatea activării/dezactivării individuale a detectoarelor la runtime. Într-un pipeline secvențial, dezactivarea unui modul ar însemna fie un if-else în corpul ciclului principal (cod fragil, cu test la fiecare cadru), fie o reconfigurare a lanțului de procesare. În arhitectura paralelă, dezactivarea înseamnă pur și simplu **a nu mai depune cadre** în coada detectorului respectiv – firul rămâne pornit, blocat pe `get()`, fără cost CPU. Reactivarea este simetrică și instantanee.

4.4.3.4 Tolerare la eșec

Dacă un detector intră într-o eroare recuperabilă (excepție tratată local, model temporar nedisponibil), pipeline-ul paralel continuă fără el; firul respectiv consumă cadrele și loghează eroarea, iar fuziunea raportează absența rezultatelor ca o lipsă tăcută. Într-un pipeline secvențial, o excepție într-un detector ar trebui prinsă explicit, iar latența cumulată a try/except-urilor ar contamina toate cadrele.

4.4.3.5 De ce nu microservicii separate

O variantă și mai distribuită ar fi un detector per proces (sau per container), cu IPC între ele. Această opțiune a fost respinsă pentru că ar fi multiplicat costul de inițializare GPU (fiecare proces și-ar fi încărcat propriile modele, depășind rapid memoria VRAM disponibilă), ar fi introdus latență suplimentară prin serializare/deserializare cadrelor și ar fi complicat semnificativ deployment-ul (cerința NF9 de portabilitate). Multi-threading în-proces oferă paralelismul logic dorit cu cost zero la nivel de date partajate – toate firele văd aceeași memorie GPU și aceleași structuri de cache, ceea ce este exact ce trebuie pentru un singur server cu GPU dedicat (vezi §4.3.3).

4.5 Proiectarea API-ului REST

Toate operațiile tranzacționale ale sistemului – autentificare, CRUD pe entități, interogări de jurnale și statistici, control al modulelor AI – sunt expuse printr-un API REST construit cu FastAPI și montat sub prefixul comun `/api`. Canalul REST este complementar canalului WebSocket (§4.6): REST-ul servește cereri punctuale, sincrone, cerere-răspuns, în timp ce WebSocket-ul servește fluxul continuu de cadre și alarme. Această secțiune prezintă endpoint-urile grupate pe categorii, convențiile de denumire, structura răspunsurilor și codurile de eroare folosite.

4.5.0.1 Convenții de denumire

API-ul respectă, în linii mari, convențiile REST: substantive la plural pentru colecții (`/api/persons`, `/api/zones`), identificatorul resursei ca segment de cale (`/api/persons/{person_id}`), iar verbul HTTP exprimă acțiunea (GET pentru citire, POST pentru creare, PUT pentru actualizare completă, PATCH pentru actualizare parțială, DELETE pentru ștergere). Sub-resursele și acțiunile derivate sunt exprimate ca segmente suplimentare (`/api/persons/{person_id}/faces`, `/api/persons/{person_id}/access-history`).

Trebuie semnalat onest că denumirile *nu* sunt perfect uniforme, fiind rezultatul unei dezvoltări iterative: controlul camerelor în timp real folosește singularul (`/api/camera/start`), în timp ce camerele persistate folosesc pluralul cu sufix (`/api/cameras-db`); alarmele sunt expuse sub `/api/alarms` (nu `/alerts`), iar statisticile sub `/api/statistics` (nu `/stats`). Aceste abateri sunt documentate ca atare și reprezintă o țintă naturală de refactorizare pentru o versiune v2 a API-ului.

4.5.0.2 Catalogul endpoint-urilor pe categorii

Autentificare.

- POST `/api/login` – autentifică un utilizator (nume + parolă) și returnează un token JWT plus datele de profil.

Stare și control camere (runtime).

- GET `/api/status` – starea curentă a sistemului: streaming activ per cameră, ce module AI sunt pornite, statistici de performanță.
- POST `/api/camera/start`, POST `/api/camera/stop`, POST `/api/camera/stop-all` – pornesc/opresc camera locală sau toate camerele.
- POST `/api/camera/add-ip`, POST `/api/camera/remove-ip` – adaugă/elimină dinamic o cameră IP prin URL.
- GET `/api/cameras/list` – lista camerelor active în memorie.

Control module AI (toggle).

- POST /api/face/toggle, /api/plate/toggle, /api/demographics/toggle, /api/fire/toggle, /api/har/toggle, /api/weapon/toggle – activează/dezactivează individual fiecare detector (cerința F7bis).
- GET /api/har/stats, GET /api/weapon/stats – statistici interne ale celor două detectoare.

Persoane.

- POST /api/persons/register – creează fișa unei persoane noi (meta-date) și returnează person_id-ul.
- GET /api/persons – listează persoanele; GET /api/persons/departments – departamentele distincte.
- GET | PUT | DELETE /api/persons/{person_id} – citire/actualizare/ștergere a unei persoane.
- POST /api/persons/{person_id}/faces – încarcă imagini faciale pentru persoană (atât la înregistrare, cât și ulterior).
- GET /api/persons/{person_id}/access-history – istoricul accesărilor persoanei.

Plăcuțe de înmatriculare.

- POST /api/plates/register; GET /api/plates; GET /api/plates/vehicle-types.
- GET | PUT | DELETE /api/plates/{plate_number}; GET /api/plates/{plate_number}/access-history.

Zone.

- GET /api/zones; POST /api/zones; PUT | DELETE /api/zones/{zone_id}.

Camere (persistate în baza de date).

- GET /api/cameras-db; POST /api/cameras-db; PUT | DELETE /api/cameras-db/{camera_id}.

Alarme.

- GET /api/alarms; GET /api/alarms/stats; GET /api/alarms/{alarm_id}.
- PATCH /api/alarms/{alarm_id} – marchează o alarmă (ex. rezolvată); POST /api/alarms/bulk-update – actualizare în masă.

Jurnale.

- GET /api/logs – jurnalul de detecții, cu filtre; GET /api/logs/vehicle – jurnalul vehiculelor.
- GET /api/logs/stats, /api/logs/timeseries, /api/logs/breakdown – agregări pentru grafice.
- GET /api/logs/export – export CSV al jurnalului filtrat.

Statistici.

- GET /api/statistics – metrice agregate la nivel de sistem.

Utilizatori.

- GET /api/users; POST /api/users; PUT | DELETE /api/users/{user_id}.
- PUT /api/users/me/password – schimbarea parolei contului propriu.

[Diagramă: **tabel** (nu diagramă) care rezumă, pe coloane, fiecare endpoint cu: metodă HTTP, cale, rol minim necesar (oricine autentificat / admin) și cerința Fx acoperită. Recomand un **longtable** întins pe categoriile de mai sus, plasat aici. Alternativ, o reprezentare a ierarhiei de resurse poate fi inclusă ca diagramă-arbore.]

4.5.0.3 Structura răspunsurilor

Răspunsurile sunt codificate JSON. Pentru operațiile de comandă (POST/PUT/PATCH/DELETE) se folosește o anvelopă consistentă cu un câmp **status** care ia valorile "success", "info" sau "error", însoțit de un câmp **message** explicativ:

```
{ "status": "success", "message": "Camera started" }
```

Răspunsul de autentificare adaugă token-ul și profilul utilizatorului:

```
{ "status": "success",  
  "token": "<JWT>",  
  "user": { "id": 1, "username": "admin",  
            "role": "admin", "full_name": "..." } }
```

Pentru operațiile de citire (GET), răspunsul este fie obiectul/colecția cerută direct, fie aceeași anvelopă cu datele atașate. Un detaliu de implementare relevant: toate răspunsurile trec printr-o funcție **convert_to_serializable** care convertește tipurile NumPy (**np.integer**, **np.floating**, **np.ndarray**) – frecvente în rezultatele modelelor AI – în tipuri native Python serializabile JSON, evitând erorile de serializare.

4.5.0.4 Coduri de eroare

API-ul folosește codurile de stare HTTP standard, ridicate prin `HTTPException` din FastAPI:

- **200 OK** – succesul operației (implicit în FastAPI).
- **401 Unauthorized** – credențiale invalide la login ("Invalid username or password"), token expirat ("Token has expired") sau token invalid ("Invalid token").
- **403 Forbidden** – utilizator autentificat dar fără rolul necesar; ridicat de dependența `require_admin` cu mesajul "Admin access required" (vezi §4.8).
- **404 Not Found** – resursă inexistentă (persoană, plăcută, zonă, alarmă cu ID inexistent).
- **422 Unprocessable Entity** – generat automat de FastAPI/Pydantic la validarea corpului cererii (câmpuri lipsă sau de tip greșit, conform modelelor `LoginRequest`, `BulkUpdateAlarmsRequest` etc.).
- **500 Internal Server Error** – excepții neașteptate (ex. eșecul deschiderii unei camere), capturate în handler și raportate cu `detail=str(e)`.

Corpul unei erori urmează formatul standard FastAPI, `{"detail": "<mesaj>"}`, ceea ce permite frontend-ului să afișeze direct mesajul în interfața în limba română.

4.6 Proiectarea canalului WebSocket

Dacă API-ul REST (§4.5) servește cereri punctuale, canalul WebSocket este responsabil de fluxul continuu de date în timp real: cadrele video procesate, adnotate cu rezultatele detectoarelor, sunt împinse de la server către toți clienții conectați. Această secțiune descrie endpoint-ul dedicat, formatul mesajelor (verificat direct în implementarea din `app.py`), cadența de transmitere și modul în care sunt tratate deconectarea și reconectarea.

4.6.0.1 Endpoint dedicat pentru streaming

Există un singur endpoint WebSocket, montat la `/ws`. La conectare, serverul *autentifică mai întâi* clientul – token-ul JWT este transmis ca parametru de interogare (`/ws?token=...`, deoarece browserele nu pot seta antete pe un WebSocket), este decodat și re-verificat față de baza de date, iar conexiunea este refuzată (cod de închidere 1008) dacă token-ul lipsește, este invalid/expirat sau aparține unui cont șters. Doar după validare serverul acceptă handshake-ul (`await websocket.accept()`) și alocă fiecărui client o **coadă proprie** (`queue.Queue`),

înregistrată în dicționarul `client_queues`. Modelul este **broadcast unu-la-mulți**: firul de fuziune (§4.4.1) nu publică într-o coadă partajată, ci difuzează fiecare cadru, prin metoda `_broadcast_frame`, în coada fiecărui client; bucla fiecărui client consumă din coada sa și trimite mesajul mai departe. Comunicarea este predominant unidirecțională, server→client; clientul nu trebuie să trimită nimic pentru a primi fluxul.

4.6.0.2 Formatul mesajelor

Fiecare mesaj este un obiect JSON serializat cu `json.dumps` și trimis prin `send_text`. Mesajul de tip cadru video are următoarea structură (construită în metoda `_encode_and_queue_frame`):

```
{
  "type": "video_frame",
  "camera_id": "CAM-01",
  "frame": "<JPEG codificat Base64>",
  "face_results": [ ... ],
  "plate_results": [ ... ],
  "fire_results": [ ... ],
  "har_results": [ ... ],
  "weapon_results": [ ... ],
  "timestamp": "2026-05-20T12:34:56.789",
  "demographics_enabled": true,
  "fire_detection_enabled": true,
  "har_enabled": true,
  "weapon_detection_enabled": true
}
```

Câmpul `frame` conține cadrul adnotat (cu bounding box-uri și etichete deja desenate de modulul de fuziune), codificat **JPEG la calitate 75** și apoi **Base64**. Pentru a reduce volumul transmis, cadrul este redimensionat la o lățime maximă de 800 px înainte de codificare. Trebuie precizat – pentru concordanță cu codul – că rezultatele detecțiilor *nu* sunt grupate într-un singur câmp `detections`, ci în **câmpuri separate per detector** (`face_results`, `plate_results`, `fire_results`, `har_results`, `weapon_results`); fiecare este o listă de obiecte cu structură specifică detectorului (de exemplu, un rezultat facial conține `name`, `confidence`, `bbox`, `age`, `gender`, `emotion`). Câmpurile booleene `*_enabled` comunică frontendului ce module sunt active, astfel încât interfața să poată ascunde/afișa straturile corespunzătoare. Toate valorile trec prin `convert_to_serializable` pentru a transforma tipurile NumPy în tipuri native serializabile JSON.

Alarmele *nu* sunt transmise printr-un tip de mesaj WebSocket separat; ele sunt persistate în baza de date și consultate de client prin endpoint-urile REST `/api/alarms` (§4.5). Captura asociată unei alarme (snapshot) este codificată tot JPEG+Base64, la calitate 60, în momentul declanșării.

[Diagramă: **diagramă de secvență UML** care arată ciclul de viață al unei conexiuni WebSocket: *autentificare (validare token)* → *handshake (accept)* →

alocare coadă proprie în `client_queues` → buclă de transmitere a mesajelor `video_frame` la ~ 30 FPS → `WebSocketDisconnect` → eliminare din dicționar.
Pe verticală: actorii Client (browser), Endpoint /ws, Coadă proprie a clientului, Firul de fuziune (`_broadcast_frame`). Plasată în §4.6.]

4.6.0.3 Cadența de transmitere

Bucula de transmitere a fiecărui client consumă din *propria coadă* în mod **non-blocking** (`get(block=False)`): dacă există un mesaj, îl trimite (`send_text(json.dumps(...))`); dacă nu, prinde `queue.Empty` și continuă. După fiecare iterație, bucla execută `await asyncio.sleep(0.03)`, ceea ce stabilește o cadență țintă de **aproximativ 30 FPS** (consistentă cu cerința NF1) și, în același timp, cedează controlul buclei de evenimente asincrone, permițând serverului să deservească în paralel ceilalți clienți și cererile REST.

4.6.0.4 Heartbeat și detectarea deconectării

Implementarea curentă **nu folosește un heartbeat la nivel de aplicație** (ping/pong explicit cu mesaje dedicate). Detectarea deconectării se bazează pe două mecanisme: (i) excepția `WebSocketDisconnect`, ridicată de FastAPI/Starlette atunci când clientul închide conexiunea, prinsă explicit în handler; și (ii) fluxul continuu de cadre, care joacă rolul unui heartbeat implicit – absența mesajelor pentru o perioadă, combinată cu eroarea la `send_text`, declanșează ieșirea din buclă. La nivel de transport, protocolul WebSocket peste TCP dispune de propriul mecanism de ping/pong de control, gestionat de bibliotecă, pe care implementarea se bazează pentru menținerea conexiunii. Indiferent de cauză, blocul `finally` asigură eliminarea cozii clientului din `client_queues` (sub protecția `client_lock`), prevenind acumularea de conexiuni moarte.

O îmbunătățire identificată pentru o versiune viitoare este adăugarea unui heartbeat aplicativ explicit (mesaj `{"type": "ping"}` periodic), util mai ales prin proxy-uri sau load-balancere care închid conexiunile inactive.

4.6.0.5 Reconectare

Reconectarea este **responsabilitatea clientului**, nu a serverului – serverul tratează fiecare conexiune ca efemeră și nu păstrează stare per client între conexiuni (este *stateless* din acest punct de vedere). La pierderea conexiunii, clientul React reia handshake-ul către /ws; deoarece serverul difuzează mereu cel mai recent cadru (mecanismul *ultimul cadru disponibil* din §5.7.2), un client reconectat își reia fluxul imediat, fără a fi nevoie de resincronizare sau de recuperarea cadrelor pierdute. Această alegere – de a nu garanta livrarea fiecărui cadru – este corectă pentru un sistem de supraveghere live, unde un cadru pierdut este irelevant atâta timp cât cel curent este actual.

4.7 Proiectarea bazei de date

Baza de date este sursa unică de adevăr a sistemului: tot ce trebuie să supraviețuiască unei reporniri – utilizatori, persoane, embedding-uri faciale, plăcuțe, zone, camere, alarme și jurnale – este persistat aici, iar structurile volatile (indexul FAISS, cache-ul de zone) sunt reconstruite din ea la pornire. Această secțiune prezintă schema reală implementată în `database_manager.py`, relațiile dintre tabele, cheile și indecșii, precum și motivarea alegerii sistemului de baze de date.

4.7.0.1 Alegerea sistemului de baze de date

Sistemul folosește **PostgreSQL**, nu **SQLite**. Decizia este motivată de mai multe trăsături folosite efectiv în schemă, indisponibile sau slabe în **SQLite**:

- tipul **tablou** (`TEXT[]`) pentru câmpul `authorized_zones` al persoanelor;
- tipul **JSONB** pentru metadatele de detecție (`detection_metadata`, `details`), cu interogare și indexare nativă;
- tipul **BYTEA** pentru stocarea embedding-urilor faciale binare;
- **concurența reală** la scriere – esențială, pentru că pipeline-ul multi-threaded (§4.4) scrie alarme și jurnale din firul de fuziune în paralel cu cererile REST ale utilizatorilor, scenariu în care blocarea la nivel de fișier a **SQLite** ar deveni un gâtuitor.

Conexiunea este configurată în `DB_CONFIG` către baza `facial_recognition` de pe `localhost`, cu credențiale dedicate (vezi și cerința NF3).

4.7.0.2 Diagrama entitate-relație

[Diagramă: **diagrama entitate-relație (ERD)** cu cele zece tabele de mai jos. Marcați cheile primare (**PK**) cu subliniere, cheile străine (**FK**) cu trimițere săgeată către tabela referită, și notați pe fiecare legătură politică **ON DELETE (CASCADE sau SET NULL)**. Atenție: relația persoană–zonă nu trece printr-un tabel de joncțiune, ci este denormalizată ca atribut tablou pe **persons** – reprezentăți-o ca atribut multi-valoare, nu ca entitate asociativă. Plasată la începutul §4.7.]

4.7.0.3 Tabelele principale

Schema reală conține zece tabele. Pentru fidelitate față de cod, mai jos sunt prezentate cu numele exacte din `database_manager.py` (care diferă pe alocuri de denumirile generice – se semnalează explicit unde e cazul).

persons – persoanele înregistrate. `id` **SERIAL** **PK**; `name`; `employee_id` (**UNIQUE NOT NULL**); `department`; `authorized_zones` **TEXT[]**; `created_at`. *Observație de proiectare:* autorizarea pe zone este **denormalizată** ca tablou de nume de zone direct pe persoană, *nu* printr-un tabel de joncțiune `person_zone_authorization`

separat. Compromisul: citire foarte rapidă a zonelor unei persoane (folosită intens de pipeline, cache-uită în `_person_zones_cache`), în schimbul pierderii integrității referențiale către `zones` (zonele sunt referite prin nume, nu prin FK).

`face_embeddings` – corespondentul `person_images` din cerință, dar stochează **embedding-uri, nu imagini**. `id` SERIAL PK; `person_id` INTEGER REFERENCES `persons(id)` ON DELETE CASCADE; `embedding` BYTEA NOT NULL (vectorul InsightFace 512-d serializat binar); `created_at`. Relația cu `persons` este **unu-la-multi** (o persoană poate avea mai multe embedding-uri, câte unul per imagine înregistrată), iar ON DELETE CASCADE asigură ștergerea embedding-urilor odată cu persoana. Acest tabel este sursa din care se reconstruiește indexul FAISS la pornire.

`access_logs` – jurnalul accesărilor de persoane. `id` SERIAL PK; `person_id` REFERENCES `persons(id)` ON DELETE CASCADE; `camera_id`; `confidence`; `detected_at`.

`license_plates` – corespondentul `plates`. `id` SERIAL PK; `plate_number` VARCHAR(20) UNIQUE NOT NULL; `vehicle_type`; `owner_name`; `owner_id`; `is_authorized` BOOLEAN DEFAULT TRUE; `registration_date`; `expiry_date`; `notes`. Cheia naturală `plate_number` este folosită ca țintă de FK de către jurnalul vehiculelor.

`vehicle_access_logs` – jurnalul accesărilor de vehicule. `id` SERIAL PK; `plate_number` cu FOREIGN KEY (`plate_number`) REFERENCES `license_plates(plate_number)` ON DELETE CASCADE; `camera_id`; `confidence`; `vehicle_type`; `detected_at`; `image_path`; `is_authorized`.

`users` – conturile de utilizator. `id` SERIAL PK; `username` UNIQUE NOT NULL; `password_hash`; `role` VARCHAR(20) DEFAULT 'user'; `full_name`; `created_at`. Parola este stocată exclusiv ca hash (cerința NF3), iar role alimentează controlul de acces (§4.8).

`alarms` – corespondentul `alerts`. `id` SERIAL PK; `camera_id` NOT NULL; `type` NOT NULL; `severity` DEFAULT 'medium'; `status` DEFAULT 'unresolved'; `description`; `snapshot` (captura JPEG+Base64 la momentul declanșării); `detection_metadata` JSONB; `created_at`; `resolved_at`; `resolved_by`; `notes`. Câmpurile `resolved_*` susțin ciclul de viață al alarmei (UC11).

`zones` – zonele de supraveghere. `id` SERIAL PK; `name` UNIQUE NOT NULL; `description`; `is_restricted` BOOLEAN DEFAULT FALSE; `created_at`.

`cameras` – registrul persistent de camere. `id` SERIAL PK; `camera_id` UNIQUE NOT NULL; `name`; `zone_id` INTEGER REFERENCES `zones(id)` ON DELETE SET NULL; `location`; `stream_url`; `camera_type` DEFAULT 'ip'; `created_at`. Politica ON DELETE SET NULL pe `zone_id` face ca ștergerea unei zone să lase camera orfană (fără zonă), nu să o șteargă.

`detection_logs` – corespondentul *logs*; istoricul complet al fiecărei detecții. `id` SERIAL PK; `camera_id` NOT NULL; `type` NOT NULL; `subject`; `confidence`; `severity`; `status`; `details` JSONB; `created_at`. Acesta este tabelul interogată de `/api/logs` și exportat ca CSV.

4.7.0.4 Chei primare și străine

Toate tabelele folosesc o cheie primară surogat `id` SERIAL (întreg auto-incrementat). Pe lângă chei primare, schema definește următoarele **chei străine** și politici de ștergere:

- `face_embeddings.person_id` → `persons.id`, ON DELETE CASCADE;
- `access_logs.person_id` → `persons.id`, ON DELETE CASCADE;
- `vehicle_access_logs.plate_number` → `license_plates.plate_number`, ON DELETE CASCADE;
- `cameras.zone_id` → `zones.id`, ON DELETE SET NULL.

Constrângeri de unicitate (UNIQUE) protejează `persons.employee_id`, `license_plates.plate_number`, `users.username`, `zones.name` și `cameras.camera_id`. *Notă:* legătura logică persoană-zonă și cea alarmă/jurnal-cameră (`camera_id` este VARCHAR, nu FK către `cameras`) nu sunt impuse prin chei străine, ci la nivel de aplicație – o decizie de proiectare care privilegiază viteza de scriere a jurnalelor în detrimentul integrității referențiale stricte.

4.7.0.5 Indexarea

Pentru a susține interogările frecvente (filtrare de jurnale, listare de alarme, căutare de plăcuțe), schema definește următorii indici secundari:

- `idx_plate_number` pe `license_plates(plate_number)`;
- `idx_detlogs_created_at` pe `detection_logs(created_at DESC)`, `idx_detlogs_type` pe `(type)`, `idx_detlogs_camera` pe `(camera_id)` – acoperă filtrele paginii de jurnale (interval de timp, tip, cameră);
- `idx_vehicle_access_time` pe `vehicle_access_logs(detected_at)`, `idx_vehicle_camera` pe `(camera_id)`;
- `idx_alarms_status`, `idx_alarms_type`, `idx_alarms_severity`, `idx_alarms_created_at` pe `alarms` – acoperă filtrarea și sortarea panoului de alarme;
- `idx_cameras_zone` pe `cameras(zone_id)`.

Indexul pe `created_at DESC` al jurnalului de detecții este deosebit de relevant: cele mai multe interogări afișează evenimentele recente în ordine descrescătoare, iar indexul descendent elimină nevoia unei sortări la fiecare cerere.

4.8 Proiectarea sistemului de control al accesului

Controlul accesului protejează operațiile sensibile ale sistemului împotriva utilizatorilor neautorizați și implementează cerința NF3. El are trei componente: un model RBAC cu două roluri, un mecanism de sesiune fără stare bazat pe JWT și o stocare sigură a parolilor. Toate sunt prezentate mai jos exact așa cum sunt implementate în `app.py` și `database_manager.py`, inclusiv limitările identificate.

4.8.0.1 Modelul RBAC

Sistemul folosește un model **Role-Based Access Control** minimal, cu două roluri stocate în coloana `users.role`:

- **admin** – acces complet: configurare, gestiunea tuturor entităților, activarea/dezactivarea modulelor AI;
- **user** – rolul implicit (DEFAULT 'user'): acces operațional la vizualizare, alarme, jurnale și statistici, fără drept de configurare.

Aplicarea rolurilor se face prin sistemul de dependențe FastAPI. Două funcții-dependență stau la baza protecției:

- `get_current_user` – decodează și validează token-ul JWT din antetul `Authorization: Bearer` și, suplimentar, *re-verifică utilizatorul față de baza de date* (prin `get_user_by_id`): un cont șters este respins cu 401 "User no longer exists", iar rolul curent din DB este preferat în locul celui din token (posibil învechit). Orice endpoint care o folosește cere doar ca utilizatorul să fie *autenticat* (oricare rol).
- `require_admin` – compusă peste `get_current_user`, verifică suplimentar `role == "admin"` și, în caz contrar, ridică `HTTPException 403` cu mesajul "Admin access required".

Astfel, nivelul de permisiune al unui endpoint este declarat la definirea lui, prin tipul dependenței injectate (`Depends(require_admin)` vs. `Depends(get_current_user)`).

4.8.0.2 Matricea de permisiuni

Tabelul 4.1 rezumă, pe module, ce poate face fiecare rol, conform dependențelor injectate efectiv în cod.

Acoperirea completă a endpoint-urilor. Spre deosebire de o versiune anterioară a sistemului, în care un set de endpoint-uri operaționale rămăseseră fără dependență de autentificare, în versiunea curentă **toate** rutele `/api` (cu excepția firească a `/api/login`) au o dependență injectată: operațiile cu efect – controlul camerelor în timp real (`/api/camera/start`, `stop`, `stop-all`, `add-ip`, `remove-ip`) – necesită `Depends(require_admin)`, iar endpoint-urile

Tabel 4.1: Matricea de permisiuni pe module și roluri.

Modul / operație	admin	user	neautentificat
Autentificare (/api/login)	✓	✓	✓
Gestiune persoane (CRUD, imagini, istoric)	✓	—	—
Gestiune plăcuțe (CRUD, istoric)	✓	—	—
Gestiune utilizatori (creare/modificare/ștergere)	✓	—	—
Toggle module AI (face/plate/fire/HAR/weapon)	✓	—	—
Creare/modificare/ștergere zone	✓	—	—
Creare/modificare/ștergere camere (DB)	✓	—	—
Control camere runtime (start/stop/add-ip/remove-ip)	✓	—	—
Stare sistem (/api/status) / listare camere active	✓	✓	—
Listare zone / camere (citire)	✓	✓	—
Statistici interne detectoare (/api/har/stats, /api/weapon/stats)	✓	✓	—
Vizualizare jurnale + export CSV (inclusiv /api/logs/vehicle)	✓	✓	—
Vizualizare + gestiune alarme (rezolvare)	✓	✓	—
Statistici (/api/statistics)	✓	✓	—
Schimbare parolă cont propriu	✓	✓	—

de citire operațională (/api/status, /api/cameras/list, /api/har/stats, /api/weapon/stats, /api/logs/vehicle) necesită `Depends(get_current_user)`. Astfel, fosta lacună prin care pornirea/oprirea camerelor era posibilă neautentificat a fost închisă.

4.8.0.3 Hashing-ul parolelor

Parolele nu sunt stocate niciodată în clar. Stocarea folosește **bcrypt** (biblioteca `bcrypt` din Python), nu `argon2`. La crearea sau actualizarea unui cont, parola este hash-uită cu `bcrypt.hashpw(password.encode(), bcrypt.gensalt())`, iar rezultatul (care include sarea generată aleator și factorul de cost încorporat în șir) este salvat în coloana `users.password_hash VARCHAR(255)`. La autentificare, verificarea se face cu `bcrypt.checkpw(password.encode(), password_hash.encode())` în metoda `authenticate_user`, care returnează datele utilizatorului *fără* câmpul `password_hash`. Folosirea lui `gensalt()` garantează o sare unică per parolă, iar costul adaptiv al `bcrypt` oferă rezistență la atacuri de tip brute-force/rainbow-table.

4.8.0.4 Gestiunea sesiunii (JWT)

Sesiunile sunt **fără stare** (*stateless*): după autentificarea reușită, serverul emite un **JSON Web Token** semnat, pe care clientul îl atașează fiecărei cereri ulterioare în antetul `Authorization: Bearer <token>`; serverul nu păstrează nicio sesiune în memorie sau în baza de date.

Parametrii concreți ai token-ului, din `app.py`:

- **Algorithm:** HS256 (HMAC-SHA256, semnătură simetrică).

- **Durată de viață:** 8 ore (`JWT_EXPIRATION_HOURS = 8`), prin câmpul `exp`.
- **Payload:** `sub` (username), `user_id`, `role`, `full_name`, `exp`, `iat`. Includerea lui `role` în token permite verificarea rolului fără o interogare suplimentară în baza de date la fiecare cerere.
- **Validare:** la decodare, `jwt.ExpiredSignatureError` produce 401 "Token has expired", iar `jwt.InvalidTokenError` produce 401 "Invalid token".

Cheia de semnare și revocarea. Cheia de semnare (`JWT_SECRET`) este **externalizată în mediul de execuție** (variabila de mediu `JWT_SECRET`); dacă nu este setată, sistemul generează o cheie aleatorie per-proces și avertizează la pornire – un fallback sigur pentru dezvoltare, dar care invalidează token-urile la fiecare repornire, motiv pentru care setarea explicită a variabilei este obligatorie pentru orice deployment real. Cât despre revocare, deși token-ul în sine este stateless, dependența `get_current_user` *re-validează la fiecare cerere* identitatea față de baza de date: ștergerea unui cont și schimbarea de rol au efect imediat (la următoarea cerere), nu doar după expirarea celor 8 ore. Singura fereastră reziduală este între emiterea token-ului și prima cerere care reflectă schimbarea; pentru cerințe stricte de revocare instantanee (ex. invalidarea unui token deja emis înainte de expirare) ar fi necesar un mecanism suplimentar (token-uri de scurtă durată cu *refresh* sau o listă neagră). Aceste aspecte sunt consemnate ca parte a analizei critice a cerinței NF3.

4.9 Proiectarea logicii de alarmare

Logica de alarmare este punctul în care detecțiile brute ale modulelor AI devin evenimente acționabile pentru operator. Ea este implementată centralizat în firul de fuziune (§4.4.1), după ce toate rezultatele unui cadru au fost agregate, și are trei responsabilități: (i) evaluarea regulilor de declanșare, (ii) atribuirea unei severități, (iii) persistarea și difuzarea alarmei cu deduplicare. Această secțiune descrie regulile exact așa cum sunt implementate în `app.py`, semnalând și diferențele față de specificația inițială.

4.9.0.1 Reguli de declanșare

O alarmă este creată prin metoda `_create_alarm_if_needed(camera_id, alarm_type, severity, description, snapshot, metadata, dedup_subject)`. Regulile efective sunt următoarele:

Persoană necunoscută (tip face). Pentru orice rezultat facial cu `name == "Unknown"`, indiferent de zonă, se creează o alarmă de tip `face` cu severitate `critical` și descrierea `"Unauthorized person detected (...%)"`.

Persoană în zonă restricționată (tip `unauthorized_zone`). Verificarea suplimentară `_check_zone_authorization` se aplică doar dacă zona camerei are `is_restricted = TRUE`. În acest caz se creează o alarmă `unauthorized_zone`

cu severitate **critical** în două situații: (a) *persoană necunoscută* în zona restricționată ("Unknown person in restricted zone '...'"), sau (b) *persoană cunoscută dar neautorizată* – recunoscută, dar al cărei `employee_id` nu apare în `authorized_zones` ale zonei ("`<nume> (<id>) in restricted zone '...' -- not authorized`"). O persoană recunoscută și autorizată nu generează nicio alarmă.

Armă detectată (tip weapon). Pentru orice rezultat din detectorul de arme se creează o alarmă **weapon**, cu severitatea preluată din rezultat (`r["severity"]`, implicit **critical**).

Foc/fum confirmat (tip fire). Se creează alarmă **fire** *doar* pentru rezultatele care au *ambele* câmpuri **confirmed** și **alert** adevărate – adică doar după confirmarea temporală pe fereastră multi-cadru (cerința F4). Severitatea este **critical** pentru clasa **fire** și **high** pentru **smoke**.

Acțiune periculoasă (tip har). Pentru orice rezultat HAR cu `class != "normal"` (fight, vandalism, fainting etc.) se creează o alarmă **har**, cu severitatea preluată din rezultat (implicit **high**) și eticheta acțiunii în descriere.

Plăcuță neautorizată (tip plate). Pentru orice rezultat ANPR cu citire OCR validă (numărul plăcuței diferit de `None`/`"`/`"UNKNOWN"`) care *nu* este regăsit pe lista de plăcuțe autorizate, se creează o alarmă **plate** cu severitate **high** și descrierea "Unauthorized vehicle `<plate>` detected (...%)", deduplicată pe numărul plăcuței. Rezultatele ANPR sunt în plus *jurnalizate* (prin `_log_detection_if_needed` cu tip **plate** și status "`authorized`" / "`unauthorized`").

Notă. Spre deosebire de regula persoanelor, alarma de plăcuță este *globală* (nu depinde de zona camerei): o plăcuță este considerată neautorizată dacă numărul ei nu apare în baza de date, indiferent de cameră (vezi §4.1.1, F3 și F7).

4.9.0.2 Captura asociată (snapshot)

Când oricare regulă se declanșează pe un cadru, se captează un *snapshot*: cadrul adnotat este redimensionat la o lățime maximă de 400 px, codificat JPEG la calitate 60 și apoi Base64, fiind atașat alarmei (câmpul **snapshot** din tabela **alarms**). Calitatea redusă este o decizie deliberată de economisire a spațiului, întrucât snapshot-ul servește doar contextualizării vizuale a alarmei în panoul operatorului.

4.9.0.3 Niveluri de severitate și mapping pe tipuri de eveniment

Severitatea este un câmp text pe tabela **alarms**, cu valoarea implicită **medium** la nivel de schemă. *Notă de concordanță cu codul:* valorile efectiv produse de logica de alarmare sunt **critical** și **high** (nu schema `info/warning/critical` din specificația inițială); **medium** rămâne doar ca implicit al coloanei, iar nivelurile

inferioare nu sunt emise de detectoare în versiunea curentă. Tabelul 4.2 prezintă maparea reală tip-eveniment → severitate.

Tabel 4.2: Maparea tipurilor de eveniment pe severitate, conform implementării.

Tip alarmă	Condiție de declanșare	Severitate
face	persoană necunoscută (oriunde)	critical
unauthorized_zone	necunoscut/neautorizat în zonă restricționată	critical
weapon	armă detectată	critical (din rezultat)
fire	foc confirmat	critical
fire	fum confirmat	high
har	acțiune ≠ normal (fight, vandalism, ...)	high (din rezultat)
plate	plăcuță neautorizată (OCR valid, negăsită)	high

4.9.0.4 Deduplicarea temporală

Pentru a nu inunda operatorul cu alarme repetate pentru același eveniment continuu, fiecare creare de alarmă trece printr-un mecanism de **cooldown**. Cheia de deduplicare este tripletul (camera_id, alarm_type, dedup_subject) – concatenat în forma "camera_id:alarm_type:subiect" – iar fereastra este alarm_cooldown_seconds = 30: dacă o alarmă cu aceeași cheie a fost creată în ultimele 30 de secunde, noua declanșare este ignorată. Verificarea și rezervarea slot-ului (actualizarea timestamp-ului) se fac sub alarm_lock, astfel încât două fire concurente să nu treacă ambele de poartă.

Componenta dedup_subject este cea care *distinge* detecțiile concurente de același tip, evitând să le colapseze într-o singură alarmă: persoanele necunoscute sunt diferențiate printr-un bucket de poziție al casetei (_bbox_position_bucket), persoanele recunoscute prin employee_id, armele prin clasă (cuțit vs. pistol), acțiunile HAR prin clasa acțiunii, iar plăcuțele prin numărul lor. Consecința – opusă unei deduplicări pe simpla pereche (cameră, tip) – este că două persoane necunoscute diferite, apărute simultan pe aceeași cameră, produc două alarme distincte, nu una. Dacă dedup_subject lipsește, mecanismul recade pe deduplicarea per (cameră, tip), ca mod implicit.

Pe lângă poarta din memorie (care se pierde la repornire), există un **backstop în baza de date**: înainte de a persista o alarmă, sistemul verifică prin get_recent_alarm dacă există deja o alarmă recentă, nerezolvată, cu aceeași cheie (inclusiv dedup_subject, păstrat în metadata), prevenind o rafală de duplicate imediat după ce serverul revine. Pentru comparație, jurnalizarea folosește propriul cooldown, mai scurt și pe subiect explicit (log_cooldown_seconds = 5).

[Diagramă: *diagramă de tip flowchart* a deciziei de alarmare pentru un cadru: intrare = rezultatele agregate → ramuri de decizie pentru fiecare tip (necunoscut? zonă restricționată? armă? foc confirmat? acțiune anormală? plăcuță neautorizată?) → atribuire severitate → construire dedup_subject → verificare cooldown 30 s pe (cameră, tip, subiect) + backstop DB → per-

sistare. Marcați faptul că plăcuța neautorizată duce atât la alarmă (*plate*, severitate *high*), cât și la jurnalizare. Plasată în §4.9.]

4.10 Proiectarea interfeței utilizator

Interfața utilizator este punctul de contact dintre operator și sistem; ea trebuie să facă vizibile, într-un mod imediat și neambiguu, fluxurile live, alarmele și starea entităților. Frontend-ul este o aplicație React (vezi §4.3.1), organizată ca un *single-page application* cu o bară laterală de navigare și un panou central de conținut care comută între ecranele descrise mai jos. Această secțiune prezintă structura de navigare, wireframe-urile ecranelor principale și principiile UX urmărite, raportate la implementarea efectivă din `frontend/src`.

4.10.0.1 Structura de navigare

Navigarea este controlată de componenta `Sidebar`, care definește nouă ecrane, fiecare cu un indicator `adminOnly` ce determină vizibilitatea în funcție de rol (mecanism aliniat cu RBAC-ul din §4.8):

- accesibile tuturor (`adminOnly: false`): *Dashboard, Alarms, Logs*;
- rezervate administratorului (`adminOnly: true`): *Person Management, Zone Management, Camera Management, Plate Management, Statistics, User Management*.

Filtrarea elementelor de meniu se face în client (`allNavItems.filter(item => !item.adminOnly || isAdmin)`), astfel încât un operator cu rol `user` nu vede deloc rutele de administrare. Aceasta este o protecție de *uzabilitate*, dublată pe server de verificările `require_admin` (protecția de securitate propriu-zisă).

4.10.0.2 Wireframe-uri pentru ecranele principale

[Diagramă: cinci *wireframe-uri* (schițe *low-fidelity*, nu capturi de ecran) pentru ecranele de mai jos. Recomand realizarea lor într-un tool de tip *Figma/Balsamiq* și includerea ca subfiguri într-o singură figură cu *subcaption*. Plasate în §4.10.]

Login. Ecran centrat, minimal: logo/iconiță, câmp utilizator, câmp parolă (mascat), buton de autentificare cu stare de încărcare (`isLoading`) și o zonă de mesaj de eroare afișată sub formularul roșu la credențiale invalide. Implementat în `Login.js`.

Dashboard live. Ecranul principal de operare: un *grid* de camere (`CameraGrid`) care afișează fluxurile primite prin WebSocket cu detecțiile suprapuse, un indicator de conexiune (`isConnected`), comutatoarele pentru modulele AI (pentru admin) și un panou de *activitate recentă* (`RecentActivity`) actualizat în timp real. Wireframe-ul trebuie să arate zona video dominantă și panoul lateral de evenimente.

Gestiune entități. Un șablon comun reutilizat pentru *Persoane*, *Plăcuțe*, *Zone*, *Camere* și *Utilizatori*: bară de acțiuni sus (căutare + buton de adăugare), tabel cu înregistrări la mijloc, acțiuni pe rând (editează/șterge) la dreapta, și un formular modal pentru creare/editare. Wireframe-ul ales ca reprezentativ poate fi *Person Management* (`PersonManagement.js`), cu sub-formularul de înregistrare cu imagini.

Alarmer. Listă de alarme (`AlarmManagement`) cu filtre pe status, tip și severitate, fiecare alarmă afișând snapshot-ul, severitatea colorată, camera și timestamp-ul, plus acțiuni de rezolvare (individual și în masă) și un câmp de note. Wireframe-ul trebuie să evidențieze codarea cromatică a severității și butoanele de triere.

Statistici. Pagină cu grafice agregate (`Statistics`) alimentate din endpoint-urile `/api/logs/stats`, `timeseries` și `breakdown`: evoluția în timp a detectiilor, distribuția pe tipuri și pe camere. Wireframe-ul arată o grilă de carduri cu grafice.

4.10.0.3 Principii UX

Feedback vizual imediat. Fiecare acțiune produce un răspuns vizibil: stările `isLoading` pe butoane în timpul cererilor, mesaje de eroare/succes, indicatorul de conexiune WebSocket (`isConnected`) pe dashboard, și suprapunerea în timp real a detectiilor peste fluxul video. Severitatea alarmelor este codată cromatic pentru identificare rapidă.

Confirmări pentru acțiuni distructive. Toate ștergerile cer o confirmare explicită prin dialog, cu mesaj care explică *consecințele*, nu doar acțiunea. De exemplu, ștergerea unei persoane avertizează că *toate datele faciale vor fi eliminate și persoana nu va mai fi recunoscută*, iar ștergerea unei zone avertizează că *camerele din ea vor rămâne neasignate, iar persoanele își vor pierde această zonă din lista autorizată*. Aceste confirmări sunt prezente în `PersonManagement`, `PlateManagement`, `ZoneManagement`, `CameraManagement` și `UserManagement`.

Filtrare consistentă. Tabelele cu volum mare de date oferă filtrare uniformă: pagina de *Alarmer* filtrează pe status/tip/severitate, pagina de *Jurnale* pe tip/status/interval de timp/căutare, ambele cu paginare. Filtrele sunt transmise ca parametri de query către endpoint-urile REST corespunzătoare, deci filtrarea se face pe server, nu în client – important pentru seturi mari de date.

Layout responsive. Stilizarea cu Tailwind folosește clase responsive (`sm:`, `md:`, `lg:`, `grid-cols-*`) în toate componentele majore, astfel încât interfața să fie utilizabilă atât pe stația fixă a dispecerului, cât și pe dispozitive mobile pentru supraveghere de la distanță (cerința NF8).

4.10.0.4 Limitări identificate față de cerințe

Verificarea codului relevă două abateri de la cerințele de uzabilitate (NF8) care trebuie consemnate onest:

- **Limba interfeței.** Cerința NF8 prevede o interfață *în limba română*, însă implementarea curentă are textele în **engleză** (etichete de meniu precum *Person Management*, mesaje de confirmare de tip **Are you sure you want to delete...**, placeholder-e de tip **Enter full name**). Internaționalizarea sau traducerea integrală în română rămâne o sarcină deschisă.
- **Sortare controlată de utilizator.** Principiul de *sortare consistentă în toate tabelele* este doar parțial îndeplinit: nu există în cod o sortare interactivă (click pe antetul coloanei), ci doar o *ordonare implicită* pe server, descrescătoare după dată (susținută de indexul `idx_detlogs_created_at` din §4.7). Adăugarea sortării interactive pe coloane este o îmbunătățire de uzabilitate identificată.

Aceste limitări sunt reluate în analiza din Capitolul 7.

Capitolul 5:

Implementarea Modulelor de Inteligență Artificială

5.1 Modulul de recunoaștere facială

Modulul de recunoaștere facială constituie componenta centrală a sistemului de control al accesului și este implementat în clasa `FacialRecognitionSystem` din fișierul `facial_recognition_system.py`. Rolul său este de a identifica, în fiecare cadru video, persoanele înregistrate anterior în baza de date și de a distinge între persoanele cunoscute (angajați) și cele necunoscute. Spre deosebire de pipeline-urile clasice de recunoaștere facială, care înlanțuie un detector de fețe, un model separat de extragere a embeddings-urilor și încă unul pentru estimarea atributelor demografice, implementarea de față consolidează toate aceste sarcini într-o singură trecere prin pachetul de modele *buffalo_s* al framework-ului InsightFace, ale cărui fundamente teoretice au fost prezentate în secțiunea 3.3.

Pipeline-ul intern al modulului are patru etape, descrise în subcapitolele care urmează: (1) detecția fețelor și extragerea embeddings-urilor cu InsightFace, (2) indexarea și căutarea vectorilor cu FAISS, (3) strategia de stabilire a identității pe baza distanței și (4) optimizarea prin omiterea analizei demografice pentru fețele deja recunoscute. La nivelul aplicației, modulul rulează într-un fir de execuție dedicat (*Thread 2*), care preia cadre dintr-o coadă de tip `queue.Queue`, apelează metoda `process_frame()` și publică rezultatele în coada de fuziune (a se vedea secțiunea 5.7).

5.1.1 Detecția fețelor și extragerea embeddings-urilor

Detecția fețelor și extragerea descriptorilor numerici (embeddings) sunt realizate de obiectul `FaceAnalysis` din InsightFace, inițializat cu pachetul de modele *buffalo_s*. Acest pachet reunește trei submodele care, în abordarea tradițională, ar fi necesitat încărcarea și apelarea separată:

- **det_500m** – detectorul de fețe, care returnează bounding box-ul și cele cinci puncte caracteristice (landmark-uri) pentru fiecare față din cadru;

- **w600k_mbf** – rețeaua de recunoaștere, o variantă MobileFaceNet antrenată pe datasetul Glint360K cu funcția de pierdere ArcFace, care produce embedding-ul facial de 512 dimensiuni;
- capul **genderage** – care estimează vârsta și genul persoanei.

Alegerea variantei `buffalo_s` (în locul, de exemplu, a variantei `buffalo_sc`) este motivată tehnic: rețeaua de recunoaștere `w600k_mbf` este identică în ambele pachete, astfel încât embeddings-urile de 512 dimensiuni rămân compatibile, însă `buffalo_s` include și capul **genderage**, indispensabil pentru analiza demografică (secțiunea 5.2). Fără acest cap, estimările de vârstă și gen ar fi returnate ca `None`.

Pentru fiecare cadru, metoda `process_frame()` apelează `self.face_app.get(frame)`, care efectuează într-o singură trecere detecția tuturor fețelor și, pentru fiecare, calculează bounding box-ul, landmark-urile, embedding-ul L2-normalizat (`normed_embedding`, vector de 512 de elemente de tip `float32`) și estimările de vârstă și gen. Bounding box-ul de tip `[x1, y1, x2, y2]` returnat de `InsightFace` este convertit în formatul `(x, y, w, h)` și ancorat în limitele cadrului de metoda statică `_bbox_xyxy_to_xywh()`, pentru a evita decupaje în afara imaginii.

5.1.1.0.1 Configurarea execuției pe GPU. Inferența `InsightFace` rulează pe GPU prin runtime-ul ONNX, providerul `CUDAExecutionProvider` fiind configurat explicit cu opțiunea `cudnn_conv_algo_search="HEURISTIC"`. Această opțiune este o decizie de implementare importantă în contextul arhitecturii multi-threaded a aplicației: strategia implicită, `EXHAUSTIVE`, măsoară empiric timpul fiecărui algoritm de convoluție folosind mecanismul de *stream capture* al CUDA. Cum `InsightFace` rulează concurent cu modelele PyTorch (YOLO pentru plăcuțe, foc și arme, respectiv SlowFast pentru acțiuni) pe același GPU, o astfel de captură în desfășurare ar întrerupe orice kernel PyTorch paralel cu eroarea *“operation not permitted when stream is capturing”*. Varianta `HEURISTIC` selectează algoritmul pe baza unor euristici, fără benchmarking și deci fără captură, eliminând coliziunile între firele de execuție.

5.1.1.0.2 Gestionarea bibliotecilor CUDA și a TensorFlow. La importul modulului, funcția `_preload_nvidia_pypi_libs()` preîncarcă explicit, prin `ctypes.CDLL(..., RTLD_GLOBAL)`, bibliotecile partajate CUDA, cuDNN și cuBLAS livrate în pachetele `pip nvidia*-cu12`, astfel încât `onnxruntime` să le poată găsi fără a necesita setarea manuală a variabilei `LD_LIBRARY_PATH`. În plus, TensorFlow (folosit de modelul de emoții) este forțat să nu folosească GPU-ul prin `set_visible_devices([], "GPU")`, deoarece menținerea lui pe GPU provoacă conflicte de context CUDA cu `onnxruntime-gpu` și `segmentation fault` la inițializare.

5.1.1.0.3 Înregistrarea persoanelor. Embeddings-urile persoanelor cunoscute sunt generate în momentul înregistrării. Metoda `register_person()`

primește o listă de imagini faciale, rulează InsightFace pe fiecare imagine (selec-tând, în cazul detecției mai multor fețe, pe cea cu aria bounding box-ului cea mai mare prin `_extract_single_face()`), extrage embedding-ul de 512 de dimen-siuni și îl persistă atât în baza de date, cât și în indexul FAISS. Pentru încărcarea de imagini prin interfața web, metoda `extract_embedding_from_image()` ac-ceptă doar imaginile care conțin exact o singură față detectabilă, evitând astfel asocierea ambiguă a unui embedding cu o persoană.

5.1.2 Indexarea cu FAISS FlatL2

Căutarea identității unei fețe presupune compararea embedding-ului ei cu toți descriptorii persoanelor înregistrate. Pentru a face această operație eficientă, sistemul folosește biblioteca FAISS, încapsulată în clasa `FaissIndex` din fișierul `faiss_index.py`. Tipul de index utilizat este ***IndexFlatL2***, care realizează o căutare *exactă* (*brute-force*) folosind distanța euclidiană L2 între vectorul de interogare și fiecare vector din index. Spre deosebire de indecșii aproximativi (precum IVFFlat), căutarea exactă nu necesită o etapă de antrenare și garan-tează găsirea celui mai apropiat vecin; aceasta este alegerea potrivită pentru dimensiunea tipică a unei baze de angajați, unde numărul de embeddings este suficient de mic încât căutarea liniară să rămână instantanee.

5.1.2.0.1 Maparea identităților. FAISS operează cu indici poziționali in-terni (0, 1, 2, ...), nu cu identificatorii persoanelor din baza de date. De aceea, clasa `FaissIndex` menține două dicționare de mapare: `id_map`, care aso-ciază fiecărui index FAISS un `person_id`, și `reverse_id_map`, care leagă fiecare `person_id` de lista de indici corespunzători (o persoană poate avea mai multe embeddings, câte unul pentru fiecare imagine de înregistrare).

5.1.2.0.2 Persistența și reconstrucția indexului. Indexul FAISS este o structură în memorie; sursa de adevăr a embeddings-urilor este baza de date PostgreSQL. În tabelul `face_embeddings`, fiecare embedding este stocat în coloana `embedding` de tip `BYTEA`, serializat cu `pickle`. La pornirea sistemu-lui, metoda `_load_embeddings_from_db()` preia toate embeddings-urile prin `get_all_embeddings()` și reconstruiește indexul de la zero apelând `rebuild_index()`. Această reconstrucție este declanșată din nou ori de câte ori se înregistrează, se editează sau se șterge o persoană, asigurând consistența între baza de date și index. Metoda de încărcare include și o protecție împotriva embeddings-urilor reziduale: dacă primul vector are o dimensiune diferită de 512 (de exemplu, em-beddings de 128 de dimensiuni dintr-un pipeline FaceNet anterior), încărcarea este abandonată, deoarece acei vectori sunt incompatibili cu modelul curent.

5.1.2.0.3 Siguranța accesului concurrent. Întrucât firul de recunoaștere facială interoghează indexul în fiecare cadru, în timp ce firele care deservesc cererile HTTP îl pot reconstrui (la înregistrarea sau ștergerea persoanelor), iar operațiile `add/search` ale FAISS nu sunt sigure pentru execuția concurrentă,

toate accesările indexului și ale dicționarelor de mapare sunt serializate cu un lock reentrant (`threading.RLock`). Caracterul reentrant este necesar deoarece `rebuild_index()` apelează intern `add_embeddings_batch()`, care preia același lock.

5.1.3 Strategia de matching

Stabilirea identității unei fețe pe baza distanței euclidiene este implementată în metoda `_match_embedding()`. Aceasta interoghează mai întâi indexul pentru cel mai apropiat vecin ($k=1$) cu un prag “infiniț”, astfel încât distanța reală până la cel mai apropiat vector să poată fi înregistrată pentru calibrare (activabilă prin variabila de mediu `FR_DEBUG`), după care aplică manual pragul de decizie configurat.

5.1.3.0.1 De la distanța L2 la similaritatea cosinus. Deoarece embeddings-urile sunt L2-normalizate, între distanța euclidiană la pătrat și similaritatea cosinus a doi vectori există relația exactă $L2^2 = 2 - 2\cos\theta$, echivalentă cu $\cos\theta = 1 - L2^2/2$. Această echivalență permite raportarea identității prin două mărimi interpretabile:

- `recognition_threshold = 1.0` – pragul aplicat distanței L2. Un prag $L2 = 1,0$ corespunde unei similarități cosinus de 0,5. Empiric, distribuția distanțelor pentru aceeași persoană se situează în intervalul $[0,6; 0,85]$ pe camera web folosită, deci un candidat cu $L2 \geq 1,0$ este respins ca necunoscut;
- `min_confidence = 0.4` – pragul aplicat încrederii, exprimată ca similaritate cosinus. Valoarea 0,4 este punctul de operare clasic “aceeași persoană” pentru ArcFace.

Dacă distanța celui mai apropiat vecin depășește `recognition_threshold`, metoda returnează `None` (față necunoscută). În caz contrar, distanța este convertită în similaritate cosinus, mărginită la intervalul $[0; 1]$ și raportată ca nivel de încredere; identificatorul persoanei este apoi rezolvat în datele complete (nume, ID angajat, departament) prin `get_person_by_id()`. În metoda `process_frame()`, o față este considerată cunoscută doar dacă există o potrivire și încrederea ei depășește `min_confidence`; pentru fiecare potrivire validă se înregistrează un eveniment de acces în baza de date prin `log_access()`.

Vizual, fețele cunoscute sunt marcate cu un chenar verde și eticheta `nume (încredere)`, iar cele necunoscute cu un chenar portocaliu și eticheta `Unknown`, prin metoda `_draw_face()`.

5.1.4 Optimizarea: omiterea analizei demografice pentru fețele cunoscute

O optimizare deliberată a pipeline-ului facial constă în condiționarea analizei demografice de rezultatul recunoașterii. În `process_frame()`, atributele demo-

grafice – vârsta și genul (produse de capul `genderage` al InsightFace) și emoția (produsă de rețeaua Mini-Xception, descrisă în secțiunea 5.2) – sunt calculate *exclusiv pentru fețele necunoscute*. Pentru o față recunoscută, câmpurile `age`, `gender`, `emotion` și `emotion_confidence` sunt setate direct la `None`.

Raționamentul are atât o componentă de proiectare, cât și una de performanță. Din punct de vedere semantic, pentru o persoană deja identificată informațiile demografice sunt redundante: identitatea, vârsta înregistrată și departamentul sunt deja cunoscute din baza de date. Din punct de vedere al performanței, omiterea evită o trecere suplimentară prin rețeaua de clasificare a emoțiilor pentru fiecare față cunoscută, în fiecare cadru – o economie semnificativă, având în vedere că această inferență rulează pe CPU. Astfel, costul analizei demografice este plătit doar atunci când aduce informație utilă, adică pentru persoanele care nu pot fi identificate și pentru care caracterizarea demografică și emoțională devine relevantă în scenariile de supraveghere.

5.2 Analiza demografică și emoțională

Pe lângă identificarea persoanelor cunoscute, sistemul caracterizează demografic și emoțional persoanele *necunoscute*, adică acele fețe care nu pot fi asociate niciunei intrări din baza de date. Această analiză produce trei atribute pentru fiecare astfel de față: vârsta estimată, genul și emoția. Implementarea este integrată în aceeași clasă `FacialRecognitionSystem` și în aceeași metodă `process_frame()` ca recunoașterea facială, evitând o trecere separată de detecție a fețelor. Așa cum s-a arătat în secțiunea 5.1.4, analiza demografică este aplicată condiționat – numai pentru fețele necunoscute – pentru că pentru persoanele identificate aceste atribute sunt redundante.

Sursa atributelor este împărțită între două modele: vârsta și genul provin de la capul `genderage` al InsightFace, deja inclus în trecerea `buffalo_s`, iar emoția este produsă de o rețea convoluțională separată, Mini-Xception, rulată pe crop-ul facial. La nivelul interfeței, analiza poate fi activată sau dezactivată global de către administrator prin endpoint-ul `/api/demographics/toggle`, care comută flag-ul `demographics_enabled`.

5.2.1 Estimarea vârstei și genului

Vârsta și genul nu necesită un model dedicat: ambele sunt produse de capul `genderage` al pachetului `buffalo_s`, în aceeași trecere InsightFace care furnizează bounding box-ul, landmark-urile și embedding-ul. Pentru fiecare obiect `Face` returnat de detector, atributele sunt extrase astfel:

- **Vârsta** – câmpul `face.age` (valoare reală) este rotunjit la cel mai apropiat întreg. Dacă atributul lipsește, vârsta este raportată ca `None`;
- **Genul** – câmpul `face.gender` este o valoare întreagă în convenția Insight-Face (1 = masculin, 0 = feminin), care este tradusă în etichetele textuale "Man", respectiv "Woman".

Această abordare nu adaugă cost de inferență suplimentar: estimările sunt deja calculate de InsightFace, fiind doar citite și formate. Tocmai disponibilitatea capului `genderage` a justificat alegerea variantei `buffalo_s` în locul variantei `buffalo_sc`, care nu îl include (secțiunea 5.1.1).

5.2.2 Recunoașterea emoțiilor cu Mini-Xception

Spre deosebire de vârstă și gen, emoția este clasificată de un model separat: Mini-Xception, o rețea neuronală convoluțională compactă livrată de pachetul Python `fer` prin fișierul de greutate `emotion_model.hdf5`. Modelul este antrenat pe datasetul FER-2013 și clasifică expresia facială într-una din șapte categorii, ordonate conform ieșirilor softmax ale modelului în tuplul `_EMOTION_LABELS`: `angry`, `disgust`, `fear`, `happy`, `sad`, `surprise` și `neutral`.

Un aspect important al implementării este faptul că API-ul implicit al pachetului `fer` este *ocolit complet*. Pachetul `fer` efectuează în mod normal propria sa detecție de fețe cu un clasificator Haar cascade înainte de a classifica emoția – o trecere redundantă, întrucât InsightFace a detectat deja toate fețele din cadru. În locul acestui flux, modelul Mini-Xception este încărcat direct prin `tf.keras.models.load_model()` (cu `compile=False`) și este alimentat cu crop-ul facial deja delimitat de bounding box-ul InsightFace. Astfel se elimină o detecție de fețe duplicată și se garantează că emoția este clasificată exact pe regiunea identificată de pipeline-ul principal.

Predicția propriu-zisă, implementată în metoda `_classify_emotion()`, constă într-o trecere înainte prin rețea, urmată de `argmax` peste cele șapte ieșiri softmax; rezultatul este perechea (etichetă emoție, scor de încredere). Dimensiunea de intrare așteptată de model este citită automat din `input_shape`, astfel încât preprocesarea (descrisă în continuare) se adaptează la geometria modelului.

5.2.3 Preprocesarea crop-ului facial

Înainte de a fi trecut prin rețeaua de emoții, crop-ul facial (extras din cadru pe baza bounding box-ului InsightFace, în format BGR) parcurge un lanț de preprocesare care reproduce fidel pipeline-ul intern al pachetului `fer`, astfel încât greutatea antrenată să primească intrări în distribuția pe care o așteaptă. Pașii, implementați în `_classify_emotion()`, sunt următorii:

1. **Conversia în tonuri de gri** – crop-ul BGR este convertit în grayscale cu `cv2.cvtColor`, deoarece modelul Mini-Xception operează pe un singur canal;
2. **Redimensionarea** – imaginea grayscale este redimensionată la dimensiunea de intrare a modelului (`_emotion_target_size`, dedusă din `input_shape`);
3. **Normalizarea** – valorile pixelilor sunt scalate din intervalul $[0; 255]$ în $[0; 1]$ (împărțire la 255), apoi mapate în intervalul $[-1; 1]$ prin transformarea $(x - 0,5) \cdot 2$;

4. **Adăugarea dimensiunilor de tensor** – matricea este extinsă la forma (1, H, W, 1) (dimensiunea de batch și canalul), formatul așteptat de un model Keras.

Întregul lanț este protejat de un bloc `try/except`: dacă, din orice motiv, crop-ul este gol sau preprocesarea eșuează, metoda returnează (`None, None`), iar fața este raportată fără emoție, fără a întrerupe procesarea celorlalte fețe din cadru.

5.2.4 Inferența pe CPU

Spre deosebire de InsightFace, care rulează pe GPU, rețeaua Mini-Xception este rulată deliberat pe CPU. Această decizie este aplicată chiar la importul modulului, unde TensorFlow este forțat să nu vadă niciun dispozitiv GPU prin `tf.config.set_visible_devices([], "GPU")`.

Motivația este dublă. În primul rând, menținerea TensorFlow pe GPU în paralel cu `onnxruntime-gpu` (folosit de InsightFace) provoacă conflicte de context CUDA și segmentation fault la initializare; izolarea TensorFlow pe CPU elimină complet această sursă de instabilitate. În al doilea rând, Mini-Xception este o rețea suficient de mică încât latența inferenței pe CPU să fie neglijabilă, iar emoția se calculează doar pentru fețele necunoscute (secțiunea 5.1.4); prin urmare, costul pe CPU este minor, iar GPU-ul rămâne dedicat integral modelelor grele (InsightFace și cele patru detectoare YOLO/SlowFast care rulează concurrent). Repartizarea modelelor pe dispozitive – modelele grele pe GPU, clasificatorul ușor de emoții pe CPU – reflectă astfel o decizie explicită de echilibrare a încărcării în arhitectura multi-threaded a aplicației.

5.3 Modulul de recunoaștere a plăcuțelor de înmatriculare

Modulul de recunoaștere a plăcuțelor de înmatriculare (ANPR – *Automatic Number Plate Recognition*) identifică și citește numerele de înmatriculare ale vehiculelor din fluxul video, le validează formatul, le caută în baza de date și decide dacă vehiculul este autorizat. Modulul este implementat în clasa `LicensePlateRecognitionSystem` din fișierul `license_plate_recognition_system.py` și rulează într-un fir de execuție dedicat (*Thread 3*).

Pipeline-ul combină un detector de obiecte (YOLOv8) cu un motor de recunoaștere optică a caracterelor (EasyOCR), între care se interpun mai mulți pași de preprocesare a imaginii și, ulterior, o etapă de stabilizare temporală a rezultatului. O provocare centrală, care a modelat întreaga implementare, este faptul că plăcuțele apar în cadru la rezoluții mici (tipic 116–151 px lățime), insuficiente pentru un OCR fiabil; din acest motiv pipeline-ul include un pas explicit de mărire (up-scaling) și o preprocesare agresivă. Cele opt subcapitole care urmează descriu, în ordinea execuției, etapele acestui pipeline.

5.3.1 Detecția plăcuțelor cu YOLOv8

Localizarea plăcuțelor în cadru este realizată de un model YOLOv8 încărcat prin clasa `YOLO` din pachetul `ultralytics`, din fișierul de greutate `models/license_plate/best.pt`. Modelul este o variantă YOLOv8 pre-antrenată specific pentru detecția plăcuțelor de înmatriculare (o singură clasă, `license_plate`), preluată ca atare; nu a necesitat antrenare suplimentară în cadrul proiectului. La inițializare, dacă există suport CUDA, modelul este mutat pe GPU.

Pentru fiecare cadru, metoda `process_frame()` apelează `predict()` cu `imgsz=640` și un prag de încredere `conf=0.25`, obținând bounding box-urile candidate. Fiecare cutie `[x1, y1, x2, y2]` este apoi *expandată* cu 20% în fiecare direcție prin funcția `expand_bbox()` (un `pad_ratio` de 0.20, mărit față de valoarea inițială de 0.05). Acest padding mai generos asigură că marginile plăcuței și o mică margine de context sunt incluse în decupaj, evitând tăierea caracterelor de la extremități înainte de OCR. Decupajul rezultat este transmis pașilor următori.

5.3.2 Up-scaling-ul plăcuțelor mici

Întrucât plăcuțele detectate sunt frecvent prea mici pentru ca OCR-ul să le citească corect, decupajul trece prin funcția `upscale_if_small()`, care îl mărește la o lățime țintă de 250 px. Logica este următoarea: dacă lățimea curentă este sub prag, se calculează un factor de scalare $scale = 250/w$, plafonat la un maxim de 3.0 (pentru a nu amplifica excesiv zgomotul), iar imaginea este redimensionată cu interpolare bicubică (`cv2.INTER_CUBIC`), aleasă pentru calitatea reconstrucției la mărire. Plăcuțele care depășesc deja lățimea țintă sunt lăsate neschimbate. Acest pas transformă, de exemplu, un decupaj de 116×33 px într-unul de cca. 250×70 px, dimensiune la care OCR-ul devine fiabil.

5.3.3 Pipeline-ul de preprocesare pentru OCR

Decupajul mărit este apoi pregătit pentru OCR de funcția `preprocess_plate()`, care aplică un lanț de patru operații tunat pentru EasyOCR pe plăcuțe:

1. **Conversie în tonuri de gri** – imaginea color este transformată în grayscale;
2. **Filtru bilateral** (`cv2.bilateralFilter` cu $d = 11$ și sigma de culoare și de spațiu de 17) – reduce zgomotul, păstrând în același timp muchiile caracterelor;
3. **Egalizare locală de histogramă CLAHE** (`clipLimit=2.0, tileGridSize=(8, 8)`) – compensează umbrele și iluminarea neuniformă;
4. **Unsharp mask** – accentuează muchiile caracterelor după mărire, construit ca diferență ponderată între imaginea originală (coeficient 1,5) și o versiune blurată Gaussian cu $\sigma = 1,0$ (coeficient $-0,5$).

O decizie deliberată este aceea de a *nu binariza* imaginea: modelul CRNN folosit de EasyOCR oferă rezultate mai bune pe imagini continue în tonuri de gri decât pe imagini cu prag aplicat. Pipeline-ul se oprește, prin urmare, la o imagine grayscale curățată și accentuată, nu la o imagine alb-negru.

5.3.4 Apelul EasyOCR

Citirea propriu-zisă a caracterelor este realizată de EasyOCR în metoda `_run_ocr()`. Apelul `reader.readtext()` este parametrizat specific pentru plăcuțe:

- `allowlist` restrânge alfabetul recunoscut la literele mari A-Z și cifrele 0-9, eliminând caracterele imposibile pe o plăcuță;
- `text_threshold=0.6`, `low_text=0.3` și `mag_ratio=1.5` reglează sensibilitatea detecției de text și mărirea internă a EasyOCR;
- `paragraph=False` dezactivează gruparea în paragrafe, nepotrivită pentru un singur rând de caractere.

Întrucât EasyOCR poate returna mai multe fragmente de text (de exemplu, grupul de litere și cel de cifre separat), fragmentele sunt sortate de la stânga la dreapta după coordonata x a colțului stânga-sus, apoi sunt curățate (păstrând doar caractere alfanumerice, transformate în majuscule) și concatenate într-un singur șir. Încrederea raportată este media aritmetică a încrederilor fragmentelor componente.

5.3.5 Fallback pe imaginea color

Preprocesarea agresivă (grayscale, filtrare, CLAHE, unsharp) îmbunătățește citirea în majoritatea cazurilor, dar în anumite condiții de iluminare poate degrada rezultatul. Pentru a acoperi aceste situații, pipeline-ul include un mecanism de *fallback*: dacă citirea de pe imaginea preprocesată este goală sau are o încredere sub 0.4, OCR-ul este reluat pe decupajul color original (nepreprocesat). Dacă această a doua citire are o încredere mai mare, ea o înlocuiește pe prima. Astfel, sistemul nu rămâne blocat pe o preprocesare care, ocazional, dăunează în loc să ajute.

5.3.6 Tracking-ul plăcuțelor între cadre

O citire dintr-un singur cadru este nesigură: OCR-ul poate confunda caractere similare (0/0, B/8) de la un cadru la altul. Pentru a obține un rezultat stabil, fiecare detecție este urmărită între cadre de clasa `PlateTracker`. Asocierea unei detecții cu un traseu existent (*track*) se face pe baza suprapunerii spațiale: se calculează *IoU* (*Intersection over Union*, prin funcția `_iou()`) între bounding box-ul curent și cele ale traseelor active, iar detecția este atribuită traseului cu IoU maxim, dacă acesta depășește pragul `iou_threshold=0.3`. Dacă niciun traseu nu se potrivește, se creează unul nou, cu un identificator unic.

Fiecare traseu păstrează un istoric de citiri recente într-o coadă mărginită (`deque` cu lungimea `history=10`) și un câmp `age` care numără de câte cadre nu a mai fost actualizat. La fiecare cadru, metoda `age_tracks()` incrementează vârsta tuturor traseelor și le elimină pe cele care depășesc `max_age=15` cadre, astfel încât vehiculele care au părăsit scena să nu mai fie urmărite.

5.3.7 Validarea formatului

Înainte ca o citire să fie luată în considerare pentru votare, ea este validată față de formatul plăcuțelor românești printr-o expresie regulată, `ROMANIAN_PLATE_RE`. Aceasta acceptă 1–2 litere de județ, urmate de 2–3 cifre și 3 litere (de exemplu, `CJ12ABC` sau, în cazul Bucureștiului, `B123ABC`). În metoda `PlateTracker.update()`, o citire este adăugată în istoricul de voturi al traseului numai dacă este nevidă, are încrederea cel puțin 0.4 și respectă acest format. Citirile malformate (rezultate, de regulă, din erori de OCR) sunt astfel filtrate înainte de a ajunge în vot, ceea ce crește robustețea rezultatului final.

5.3.8 Votarea temporală

Rezultatul stabil al unui traseu este decis printr-un *vot majoritar* pe istoricul de citiri valide. Logica, implementată tot în `PlateTracker.update()`, se activează numai după ce traseul a acumulat cel puțin `min_votes=3` citiri. Voturile sunt agregate într-un `Counter` în care fiecare text candidat acumulează suma încrederilor citirilor sale; textul cu scorul total maxim este selectat, iar dacă numărul de citiri care îl susțin atinge pragul `min_votes`, el devine rezultatul “emis” al traseului (`emitted`), cu o încredere egală cu media încrederilor voturilor sale.

Doar după ce un traseu produce o citire stabilizată sistemul efectuează căutarea în baza de date: `lookup_owner_by_plate()` caută proprietarul, iar prezența unei intrări marchează vehiculul ca autorizat. Plăcuțele nestabilizate sau necunoscute sunt raportate ca `UNKNOWN` / “Unknown car”, iar vizual vehiculele autorizate sunt încadrate cu verde, cele neautorizate cu roșu. Acest mecanism de votare temporală asigură că o plăcuță este declarată și verificată doar atunci când mai multe cadre consecutive sunt de acord asupra aceluiași text valid, eliminând citirile tranzitorii eronate.

5.4 Modulul de detecție foc și fum

Modulul de detecție a focului și a fumului identifică în fluxul video apariția flăcărilor și a fumului, declanșând alerte timpurii în scenariile de incendiu. Este implementat în clasa `FireDetectionSystem` din fișierul `fire_detection_system.py` și rulează într-un fir de execuție dedicat (*Thread 4*).

Provocarea principală a acestui modul nu este detecția în sine, ci controlul falselor pozitive: focul și mai ales fumul sunt clase notorii de predispuse la confuzii (pereți gri, nori, abur, reflexii calde, apusuri de soare). O detecție brută a unui model YOLO, folosită direct ca alarmă, ar genera un volum inacceptabil

de alarme false. Din acest motiv, implementarea adaugă peste model un *pipeline de filtrare în cinci niveluri*, care transformă detecțiile candidate în alerte numai după ce trec, în ordine, prin toate filtrele. Subcapitolele care urmează descriu modelul de bază, cele cinci niveluri de filtrare și, în final, semnificația flag-urilor `confirmed` și `alert` atașate fiecărei detecții.

5.4.1 Modelul YOLOv10 pre-antrenat

Modelul de bază este `TommyNgx/YOLOv10-Fire-and-Smoke-Detection`, o variantă YOLOv10 publicată pe HuggingFace și antrenată specific pentru cele două clase de interes: `Fire` și `Smoke`. Modelul este folosit ca atare, fără antrenare suplimentară, fiind încărcat prin clasa `YOLO` din pachetul `ultralitics` din fișierul de greutate `models/fire_and_smoke/best.pt`. Numele claselor sunt normalizate la litere mici (`fire`, `smoke`) imediat după încărcare, pentru a fi folosite consistent în restul pipeline-ului.

La fiecare cadru, metoda `process_frame()` rulează `model.predict()` cu un prag de încredere egal cu *cel mai mic* dintre pragurile per clasă (și un IoU de NMS de 0.45). Această alegere este deliberată: modelul este lăsat să returneze toți candidații peste pragul minim, iar filtrarea fină per clasă este aplicată ulterior, explicit, de pipeline – ceea ce dă control complet asupra deciziei de alertă.

5.4.2 Pipeline-ul de filtrare în cinci niveluri

Fiecare detecție candidată returnată de YOLOv10 parcurge, în ordine, cinci niveluri de filtrare. O detecție trebuie să treacă de toate cele cinci niveluri pentru a deveni o alertă; eșecul la oricare nivel o elimină (sau, în cazul confirmării temporale, o reține ca neconfirmată încă). Statisticile interne (`rejected_by_size`, `rejected_by_color` etc.) contorizează câte detecții sunt respinse la fiecare nivel.

5.4.2.1 Praguri de încredere per clasă

Primul nivel aplică un prag de încredere *diferențiat pe clasă*: 0.40 pentru `fire` și 0.55 pentru `smoke`. Pragul mai sever pentru fum reflectă faptul că acesta este semnificativ mai predispus la false pozitive (suprafețe gri, nori, abur) decât focul, care are o semnătură vizuală mai distinctivă. O detecție a cărei încredere este sub pragul clasei sale este eliminată imediat.

5.4.2.2 Filtre de dimensiune

Al doilea nivel verifică plauzibilitatea dimensiunii bounding box-ului, raportată la aria cadrului. Sunt respinse atât detecțiile neglijabil de mici (sub 0.3% din cadru – tipic zgomot sau reflexii punctuale), cât și cele excesiv de mari (peste 85% din cadru – tipic o clasificare greșită care “acoperă” aproape întreaga imagine). Doar detecțiile cu o arie relativă în intervalul [0,003; 0,85] trec mai departe.

5.4.2.3 Verificarea cromatică HSV

Al treilea nivel este o verificare cromatică euristică, realizată în spațiul de culoare HSV de metoda `_color_plausible()`, care exploatează faptul că focul și fumul au semnături de culoare caracteristice:

- **Foc** – regiunea trebuie să conțină o fracție semnificativă de pixeli “calzi”, saturați și luminoși: nuanțe (*hue*) în jurul roșu-portocaliu-galben (care se încadrează la $h \leq 25$ sau $h \geq 160$, deoarece nuanțele calde se “înfășoară” în jurul valorii 0), cu saturație $s \geq 90$ și luminozitate $v \geq 120$. Se cere ca cel puțin 12% din pixeli să fie calzi;
- **Fum** – regiunea trebuie să fie de saturație redusă și luminozitate moderată: cel puțin 55% din pixeli trebuie să fie cenușii (saturație $s \leq 60$ și luminozitate $v \geq 40$, pentru a respinge zonele complet negre). În plus, deviația standard a luminozității trebuie să fie cel puțin 6, condiție care respinge suprafețele perfect uniforme (cer senin, perete), unde fumul real ar prezenta variație.

Dectiile care nu îndeplinesc condiția cromatică a clasei lor sunt respinse, eliminând o mare parte din falsele pozitive care ar trece de primele două niveluri.

5.4.2.4 Confirmarea temporală prin tracking IoU

Al patrulea nivel cere persistentă în timp: o detecție devine *confirmată* numai dacă apare în mod consistent pe cel puțin 6 cadre consecutive. Pentru a urmări o aceeași detecție de la un cadru la altul, metoda `_update_frame_history()` asociază bounding box-ul curent cu intrările din istoricul de cadre pe baza suprapunerii spațiale ($\text{IoU} \geq 0,3$, calculat de `_compute_iou()`), restrânsă la aceeași clasă. Dacă se găsește o potrivire, contorul de cadre consecutive al intrării este incrementat; altfel se creează o intrare nouă, cu contor 1. Intrările care nu mai sunt observate într-un cadru sunt expirate (eliminate), întrerupând astfel seria. Acest mecanism elimină detecțiile tranzitorii de un singur cadru (sclipiri, artefacte de compresie), care nu pot reprezenta un incendiu real.

5.4.2.5 Cooldown de alertă pe locație

Al cincilea nivel previne inundarea cu alerte repetate pentru același eveniment. Odată ce o detecție este confirmată, se generează o alertă numai dacă a trecut suficient timp de la ultima alertă pentru *aceeași locație*. Locația este aproximată printr-o cheie grosieră, construită din clasa detecției și din coordonatele colțului împărțite la 40 (`class_name_x1//40_y1//40`), astfel încât detecții ușor deplasate să fie tratate ca aceeași sursă. Intervalul minim de cooldown între două alerte la aceeași locație este de 3 secunde. Astfel, un incendiu persistent generează o alertă inițială și apoi reamintiri rare, în loc de o avalanșă de alerte la fiecare cadru.

5.4.3 Flag-urile *confirmed* și *alert*

Fiecare detecție returnată de `process_frame()` poartă două flag-uri booleene care codifică rezultatul pipeline-ului de filtrare și care au roluri distincte:

- **confirmed** – adevărat când detecția a trecut de primele patru niveluri și a atins pragul de persistență temporală (cel puțin 6 cadre consecutive). O detecție confirmată este considerată un eveniment real și este *desenată* pe cadru; candidații neconfirmați nu sunt afișați, evitând clipirea vizuală a casetelor nesigure;
- **alert** – adevărat doar în cadrul în care o detecție confirmată trece și de nivelul de cooldown, marcând momentul precis în care trebuie generată o *alarmă* (de exemplu, înregistrarea în baza de date și notificarea prin WebSocket). Vizual, o detecție cu **alert** este desenată cu un chenar mai gros, iar la nivel de cadru se afișează un banner “FIRE ALERT”.

Distincția dintre cele două flag-uri separă astfel *starea* (există foc/fum confirmat în acest cadru?) de *evenimentul* (trebuie declanșată acum o alarmă?), permițând afișarea continuă a detecțiilor confirmate fără a genera alarme redundante.

5.5 Modulul de detecție a armelor (model fine-tuned propriu)

Spre deosebire de modulele anterioare, care folosesc modele pre-antrenate preluate ca atare, modulul de detecție a armelor se bazează pe un model YOLOv8 *fine-tuned* în cadrul proiectului, pe un set de date construit special pentru această sarcină. Motivul este lipsa unui model public suficient de robust pentru detecția generică a armelor în context de supraveghere. Această secțiune descrie întregul proces, de la deciziile de proiectare (o singură clasă, alegerea modelului de bază), prin construcția și unificarea setului de date, până la parametrii de antrenare și analiza rezultatelor. Pipeline-ul de antrenare este organizat în trei scripturi numerotate, rulate în ordine: `01_download_datasets.py`, `02_relabel_and_merge.py` și `03_train_yolo.py`.

5.5.1 Justificarea unei singure clase *weapon*

Modelul este antrenat să recunoască o *singură* clasă generică, **weapon**, care acoperă deopotrivă arme de foc (pistoale, puști), arme albe (cuțite, macete) și obiecte contondente (bâte de baseball). Decizia de a colapsa toate tipurile de arme într-o singură clasă este motivată practic: în contextul unui sistem de supraveghere, ceea ce contează este *prezența* unei arme în cadru, nu categoria ei exactă. Această simplificare reduce problema de clasificare la una de detecție binară (armă / non-armă), permite modelului să generalizeze mai bine pe exemple noi și elimină confuziile inutile între subtipuri de arme, care oricum ar declanșa aceeași alertă.

5.5.2 Selectarea modelului de bază

Modelul de bază ales pentru fine-tuning este **YOLOv8m** (varianta *medium*), pre-antrenat pe datasetul COCO. Această variantă oferă un echilibru bun între acuratețe și viteză de inferență: este suficient de expresivă pentru a învăța semnătura vizuală variată a armelor, dar suficient de ușoară pentru a rula în timp real, alături de celelalte modele, pe același GPU. Fine-tuning-ul pornește de la greutățile COCO (`pretrained=True`), beneficiind astfel de reprezentările vizuale generice deja învățate, peste care se adaptează detectorul la noua clasă.

5.5.3 Dataseturile utilizate

Întrucât niciun dataset public unic nu acoperea satisfăcător diversitatea de arme, condiții de iluminare și perspective dorite, au fost combinate două seturi de date publice, descărcate automat de scriptul `01_download_datasets.py`:

- **Dangerous Items** (Zenodo) – 8.478 de imagini, cu cinci clase originale, toate arme: `machete`, `knife`, `baseball_bat`, `rifle` și `gun`. Imaginile acoperă scene diverse, de la arme prezentate izolat până la arme ținute în mână;
- **SOHAS Weapon Detection** (Roboflow) – 5.858 de imagini, cu șase clase originale: `pistol`, `knife`, `smartphone`, `billete` (bancnote), `monedero` (portofel) și `tarjeta` (card). Ultimele patru clase sunt obiecte non-armă care pot fi ușor confundate cu arme și ajută modelul să învețe să le diferențieze.

După combinare, datasetul final conține 14.336 de imagini și 12.592 de bounding box-uri, oferind o diversitate suficientă pentru antrenarea unui detector robust.

5.5.4 Preprocesarea și unificarea

Cele două dataseturi folosesc scheme de etichetare diferite, astfel încât a fost necesară o etapă de preprocesare pentru unificarea lor într-o structură YOLO coerentă, realizată de scriptul `02_relabel_and_merge.py`. Operațiile principale sunt:

- **Relabel** – toate clasele de arme din ambele dataseturi sunt remapate la clasa unică 0 (`weapon`). În cazul Zenodo, toate cele cinci clase sunt arme, deci toate sunt păstrate; în cazul Roboflow, doar `pistol` și `knife` sunt păstrate, iar clasele non-armă (`smartphone`, `billete`, `monedero`, `tarjeta`) sunt *eliminate* din etichete;
- **Merge** – cele două dataseturi sunt unite într-o singură structură YOLO cu trei partiții (`train`, `valid`, `test`); partiția `val` a sursei Zenodo este redenumită `valid` pentru consistență;

- **Negative samples** – imaginile fără adnotări (în care nu apare nicio armă) sunt păstrate ca exemple negative. Ele învață modelul ce *nu* este o armă și reduc rata de false pozitive la inferență, mai ales pentru obiectele care seamănă cu armele (cazul imaginilor cu telefoane sau portofele din Roboflow, ale căror etichete au fost golite).

La final, scriptul generează automat un fișier `data.yaml` cu structura standard YOLO, declarând o singură clasă (`nc: 1, names: {0: weapon}`).

5.5.5 Strategia de denumire (*zen_ / rf_*)

O problemă subtilă la unirea a două dataseturi este coliziunea numelor de fișiere: ambele surse pot conține, de exemplu, o imagine numită `0001.jpg`, care s-ar suprascrie reciproc la copierea în directorul comun. Pentru a o evita, fiecărei imagini și etichete i se adaugă un prefix în funcție de sursă: **zen_** pentru imaginile din Zenodo și **rf_** pentru cele din Roboflow. Astfel, perechile imagine–etichetă rămân corelate (același prefix și același nume de bază), iar fișierele celor două surse coexistă fără pierderi în arborele `train/valid/test` comun.

5.5.6 Parametrii de antrenare

Antrenarea propriu-zisă, realizată de scriptul `03_train_yolo.py`, a folosit următorii parametri principali:

- **Epoci:** 100 de epoci planificate, cu *early stopping* (`patience=20`) – oprire automată dacă metrica de validare nu se mai îmbunătățește timp de 20 de epoci;
- **Batch size:** 16 imagini per batch;
- **Rezoluția de intrare:** 640×640 pixeli, valoarea standard pentru YOLOv8;
- **Optimizator:** AdamW, cu learning rate inițial $lr_0 = 0,001$ și factor final $lr_f = 0,01$, conform unei programări cosinus a ratei de învățare (`cos_lr=True`), și `weight_decay = 0,0005`;
- **Warmup:** 3 epoci de warmup pentru stabilizarea gradientilor la începutul antrenării.

Scriptul include și o rutină de selecție automată a GPU-ului cu cea mai multă memorie liberă (`pick_gpu()`, via `pynvml` sau, în lipsa lui, `nvidia-smi`). Antrenarea a fost realizată pe un GPU NVIDIA A100.

5.5.7 Augmentările folosite

Pentru a crește robustețea modelului și a reduce overfitting-ul, în timpul antrenării au fost aplicate multiple transformări de augmentare a datelor:

- **Variații cromatice HSV** – nuanță ($\pm 0,015$), saturatie ($\pm 0,7$) și luminositate ($\pm 0,4$), pentru a simula condiții de iluminare diferite;
- **Transformări geometrice** – rotație până la 10° , translație de 10%, scalare de 50% și oglindire orizontală (`flip_lr=0.5`);
- **Mosaic** (`mosaic=1.0`) – combinarea a patru imagini într-una singură, expunând modelul la contexte și scale variate;
- **MixUp** (`mixup=0.1`) – amestecarea a două imagini, ca regularizare suplimentară.

5.5.8 Rezultatele antrenării

Antrenarea a parcurs toate cele 100 de epoci (early stopping nu s-a declanșat). Metricile pe partiția de validare la finalul antrenării sunt sintetizate în tabelul 5.1.

Tabel 5.1: Metricile finale de validare ale modelului de detecție a armelor.

Metrică	Valoare
Precizie (<i>precision</i>)	0,937
Sensibilitate (<i>recall</i>)	0,894
mAP@0,5	0,947
mAP@0,5:0,95	0,700

Modelul atinge o precizie de 93,7% și o sensibilitate de 89,4%, cu un mAP@0,5 de 0,947. Valoarea mAP@0,5:0,95 (media mAP peste praguri IoU de la 0,5 la 0,95) este de 0,700, indicând o localizare bună, deși mai puțin precisă la pragurile IoU foarte stricte – comportament tipic pentru detecția unor obiecte cu forme și orientări foarte variate, cum sunt armele. Pe lângă greutatea, antrenarea a produs și artefacte de diagnostic (curbele precizie-sensibilitate, matricea de confuzie, curbele de pierdere), salvate în directorul de ieșire al rulării.

5.5.9 Discuție asupra rezultatelor

Valorile obținute sunt solide pentru o sarcină dificilă: detecția unei clase vizual eterogene (o armă de foc, un cuțit și o bătă de baseball au foarte puține trăsături comune), pe un dataset combinat din surse cu stiluri de adnotare diferite. Diferența dintre precizie (93,7%) și sensibilitate (89,4%) arată că modelul este ușor mai conservator: ratează unele arme, dar generează relativ puține detecții false – un compromis dezirabil pentru a evita alarmele false într-un sistem de securitate.

Acest comportament este întărit la inferență de o decizie de proiectare la nivelul modului de rulare (`WeaponDetectionSystem`): deși modelul produce candidați peste un prag de încredere permisiv (0,3), o detecție este reținută doar dacă depășește un post-filtru strict de încredere (0,7), iar alerta este confirmată

temporal pe mai multe cadre consecutive (mecanism analog celui de la detecția de foc, secțiunea ??). Astfel, sensibilitatea brută a modelului este deliberat schimbată pe o precizie operațională mai mare, prioritizând reducerea falselor alarme. Aportul exemplurilor negative (telefoane, portofele, carduri din SOHAS, cu etichete golite) se reflectă tocmai în această precizie ridicată, modelul fiind expus explicit la obiecte care pot fi confundate cu arme.

5.6 Modul de recunoaștere a acțiunilor umane (fine-tuning SlowFast)

Cel mai complex modul de inteligență artificială al sistemului este cel de recunoaștere a acțiunilor umane (HAR – *Human Action Recognition*), care clasifică *secvențe video* (nu cadre izolate) în trei categorii comportamentale: **normal** (activitate obișnuită), **fight** (lupte, altercații fizice) și **vandalism** (acte de distrugere intenționată a proprietății). Spre deosebire de detectoarele de obiecte, care operează pe un singur cadru, recunoașterea acțiunilor necesită modelarea dimensiunii temporale – aceeași imagine poate aparține atât unei îmbrățișări, cât și unei altercații, distincția fiind dată de mișcare.

Acest modul reprezintă, alături de detectorul de arme, una dintre cele două componente antrenate în cadrul proiectului. Antrenarea (fine-tuning) a fost realizată în proiectul separat `security_system`, iar modelul rezultat este consumat de aplicația principală ca un fișier checkpoint (`best_model.pth`), încărcat la rulare de clasa `HumanActionRecognitionSystem` din `har_system.py` (*Thread 5*). Subcapitolele care urmează descriu alegerea arhitecturii, construcția setului de date (inclusiv crearea de la zero a unui dataset propriu pentru clasa **vandalism**), procesul de fine-tuning, parametrii și rezultatele, iar la final modul de integrare în aplicație.

5.6.1 Justificarea alegerii SlowFast R50

Arhitectura aleasă este **SlowFast R50**, dezvoltată de Facebook AI Research, ale cărei fundamente au fost prezentate în secțiunea 3.6.2. Ea se bazează pe două căi de procesare paralele: o cale *Slow*, care procesează puține cadre la rezoluție temporală redusă și captează informația semantică spațială, și o cale *Fast*, care procesează multe cadre și captează dinamica mișcării, cele două fiind unite prin conexiuni laterale. Această separare explicită a aspectului spațial de cel temporal o face deosebit de potrivită pentru distingerea acțiunilor pe baza mișcării – exact problema de față.

Varianta R50 (cu backbone ResNet-50) oferă un compromis bun între capacitate și cost computațional, permițând inferența în timp real alături de celelalte modele. În plus, modelul este disponibil pre-antrenat pe datasetul Kinetics-400 (prin PyTorchVideo / PyTorch Hub), ceea ce oferă un punct de pornire solid: reprezentările video generice învățate pe 400 de clase de acțiuni pot fi adaptate prin fine-tuning la cele trei clase ale problemei, cu un set de date mult mai mic decât ar fi necesar pentru antrenarea de la zero.

5.6.2 Construcția setului de date

Setul de date pentru fine-tuning a fost construit din trei surse, câte una pentru fiecare clasă. Structura este de tip folder-per-clasă (`data/train/<clasă>/` și `data/test/<clasă>/`), iar clasele sunt descoperite automat din numele folderelor de către `configs/config.py`, cu convenția ca eticheta `normal` să primească întotdeauna indexul 0.

5.6.2.1 Clasa *normal*

Clasa `normal` conține videoclipuri de supraveghere cu activitate cotidiană, fără incidente, preluate din datasetul public **RWF-2000** (disponibil pe Kaggle). Aceste secvențe oferă modelului o referință pentru ceea ce înseamnă comportament obișnuit, esențială pentru a delimita clasele de interes (`fight`, `vandalism`) de fundalul normal.

5.6.2.2 Clasa *fight*

Clasa `fight` conține videoclipuri cu lupte și altercații fizice, preluate tot din datasetul **RWF-2000**, care include scene de confruntare fizică între persoane, capturate de camere de supraveghere. Folosirea aceluiași dataset pentru clasele `normal` și `fight` asigură un stil vizual consistent (rezoluție, unghiuri, calitate a imaginii) între cele două, astfel încât modelul să învețe diferența de *acțiune*, nu de domeniu vizual.

5.6.2.3 Clasa *vandalism* – dataset propriu

5.6.2.3.1 Justificarea creării unui dataset propriu. Spre deosebire de primele două clase, pentru `vandalism` nu există un dataset public adecvat de videoclipuri de supraveghere. Din acest motiv, a fost necesară construirea unui set de date propriu, de la zero, printr-un proces în trei etape descris în continuare.

5.6.2.3.2 Etapa 1: Scraping automat YouTube. A fost dezvoltat un script Python dedicat care folosește biblioteca `yt-dlp` pentru a căuta și colecta automat link-uri către videoclipuri candidate de pe YouTube. Scriptul generează sute de interogări de căutare variate, în mai multe limbi, acoperind acte specifice de vandalism (graffiti, spargere de geamuri, distrugere de mașini, intrare prin efracție etc.) combinate cu termeni asociați camerelor de supraveghere. În urma acestui proces au fost colectate aproximativ 4.800 de link-uri către potențiale videoclipuri.

5.6.2.3.3 Etapa 2: Filtrare manuală. Toate cele aproximativ 4.800 de videoclipuri au fost vizualizate și evaluate manual, pentru a selecta doar clipurile care conțin efectiv acte de vandalism filmate de camere de supraveghere, eliminându-le pe cele irelevante. În urma filtrării au rămas peste 600 de videoclipuri valide. Această etapă, deși laborioasă, este esențială pentru calitatea datasetului: garantează că modelul învață din exemple reale și relevante.

5.6.2.3.4 Etapa 3: Editare în OpenShot. Videoclipurile selectate au fost editate individual cu software-ul OpenShot Video Editor, pentru a extrage doar segmentele relevante. Fiecare clip a fost decupat astfel încât să conțină secvența propriu-zisă a actului de vandalism, eliminând porțiunile irelevante (intro-uri, text suprapus, secvențe fără activitate).

5.6.2.3.5 Setul final. Setul de date final pentru clasa `vandalism` conține **614 videoclipuri** cu acte de vandalism, acoperind o gamă largă de acțiuni: graffiti / street art, spargere de geamuri și vitrine, distrugere de autoturisme (zgâriere, lovire), intrare prin efracție, distrugere de mobilier urban, aruncare cu obiecte și alte forme de distrugere intenționată a proprietății.

5.6.3 Pregătirea modelului pentru fine-tuning

Pregătirea modelului pornește de la SlowFast R50 pre-antrenat pe Kinetics-400, al cărui cap de clasificare original – un strat `Linear(2304, 400)` corespunzător celor 400 de clase Kinetics – este înlocuit cu un cap nou, adaptat problemei: o secvență `Dropout(0.5)` urmată de un `Linear(2304, 3)`, pentru cele trei clase. Această înlocuire este realizată în `build_slowfast_model()` din `models/slowfast_model.py`.

5.6.3.0.1 Formatul intrării (contractul SlowFast). Fiecare videoclip este transformat într-o pereche de tensori, câte unul pentru fiecare cale: calea Slow primește 8 cadre, iar calea Fast 32 de cadre, ambele decupate spațial la 224×224 pixeli. Cadrele sunt eșantionate uniform dintr-un clip corespunzător unei durate de 2.0 secunde. Aceste valori sunt fixate în `configs/config.py` și trebuie respectate identic la inferență (în aplicație), altfel modelul ar primi intrări incompatibile cu cele pe care a fost antrenat.

5.6.4 Parametrii de antrenare

Antrenarea, implementată în `train.py`, folosește o strategie de fine-tuning în *două faze*, gândită pentru a adapta modelul la noile clase fără a distruge reprezentările pre-antrenate:

- **Faza 1 – backbone înghețat** (primele 5 epoci): toți parametrii backbone-ului sunt înghețați (`requires_grad=False`), antrenându-se *doar* capul de clasificare nou. Astfel, capul aleator inițializat se stabilizează fără a perturba caracteristicile vizuale învățate pe Kinetics-400;
- **Faza 2 – fine-tuning complet:** backbone-ul este dezghețat și întregul model este antrenat, dar cu *rate de învățare diferențiate* – backbone-ul primește o rată de 10 ori mai mică decât capul (`lr_backbone_factor=0.1`), pentru a-l ajusta fin fără a-l deteriora.

Alți parametri și tehnici folosite, conform `config.py` și `train.py`:

- **Optimizator:** AdamW, rată de învățare 10^{-3} , `weight_decay` = 10^{-4} , cu o programare de tip StepLR (reducere de $10\times$ la fiecare 10 epoci);
- **Funcția de pierdere:** CrossEntropyLoss cu *label smoothing* de 0,1, pentru a reduce supraîncrederea modelului;
- **Dezechilibrul de clase:** tratat printr-un WeightedRandomSampler, care eșantionează clasele invers proporțional cu frecvența lor, astfel încât fiecare clasă să fie reprezentată echilibrat în batch-uri;
- **Tehnici de stabilizare:** antrenare în precizie mixtă (AMP, prin GradScaler și autocast), *gradient clipping* la norma maximă 5,0 și *early stopping* (răbdare de 7 epoci) bazat pe macro-F1 de validare;
- **Augmentare spațială:** oglindire orizontală, *color jitter* și *random crop*.

Din cele 30 de epoci planificate, antrenarea s-a oprit la epoca 28 prin early stopping, cele mai bune rezultate (cele salvate ca `best_model.pth`) fiind obținute la epoca 21. Antrenarea a folosit 262 de batch-uri pe epocă la antrenare și 56 la validare, cu o durată medie de aproximativ 15 minute per epocă.

5.6.5 Rezultatele

Metricile globale obținute pe setul de validare de către modelul cel mai bun (epoca 21) sunt sintetizate în tabelul 5.2. Pierderea finală de antrenare a fost de 0,3065, iar cea de validare de 0,4306.

Tabel 5.2: Metricile globale de validare ale modelului de recunoaștere a acțiunilor.

Metrică	Valoare
Acuratețe (<i>accuracy</i>)	0,9433
Macro Precision	0,9478
Macro Recall	0,9436
Macro F1-Score	0,9456

Modelul atinge o acuratețe globală de 94,33% și un macro F1-score de 94,56%, valori foarte bune pentru o problemă de clasificare a acțiunilor pe trei clase, în condiții de supraveghere reale.

5.6.6 Analiza per clasă

Rezultatele detaliate pe fiecare clasă sunt prezentate în tabelul 5.3.

Se remarcă faptul că tocmai clasa **vandalism** – cea pentru care a fost construit datasetul propriu – obține cele mai bune rezultate, cu o precizie de 98,2% și un F1-score de 0,965. Acest rezultat validează efortul de creare a datasetului custom: clipurile selectate și editate manual sunt suficient de curate și de reprezentative pentru ca modelul să învețe robust această clasă. Clasele **normal**

Tabel 5.3: Metricile per clasă ale modelului de recunoaștere a acțiunilor.

Clasă	Precision	Recall	F1-Score	Suport
normal	0,920	0,947	0,933	169
fight	0,942	0,935	0,939	155
vandalism	0,982	0,949	0,965	117

și **fight** obțin de asemenea scoruri ridicate (F1 de 0,933, respectiv 0,939), confuziile reziduale fiind, în mod așteptat, preponderent între ele – ambele provin din același dataset RWF-2000 și pot conține secvențe ambigue la granița dintre activitate agitată și altercație.

5.6.6.0.1 Integrarea în aplicație. La rulare, modelul antrenat este folosit de `HumanActionRecognitionSystem`, care nu poate clasifica un singur cadru, ci are nevoie de o secvență. De aceea, sistemul acumulează cadrele primite într-un *ring buffer* (`deque`) și rulează inferența doar o dată la fiecare 30 de cadre (`clip_interval_frames`), atunci când bufferul conține destule cadre. La fiecare inferență, se eșantionează uniform până la 64 de cadre din buffer, se construiește perechea *slow/fast* (8/32 cadre, 224×224) exact ca la antrenare, se aplică **softmax** peste logits și se reține clasa cu probabilitatea maximă. O acțiune non-normală este raportată ca alertă doar dacă încrederea depășește pragul `confidence_threshold` (0,5), cu un *cooldown* de alertă de 5 secunde pentru a evita notificările repetate. Predicția curentă este păstrată între inferențe, astfel încât adnotarea afișată pe flux rămâne coerentă chiar și pe cadrele dintre două rulări de inferență.

5.7 Fuziunea rezultatelor pe cadru

Cele cinci module de inteligență artificială descrise anterior nu rulează izolat, ci sunt orchestrate de aplicația principală (`app.py`) într-un *pipeline multi-threaded* care procesează același flux video în paralel și apoi recombina rezultatele într-un singur cadru adnotat. Această secțiune descrie modul în care detecțiile provenite de la modele diferite, care rulează pe fire de execuție separate și se termină la momente diferite, sunt sincronizate, suprapuse pe cadru, transformate în alarme și difuzate către clienți.

Decizia arhitecturală fundamentală este aceea de a folosi *fire de execuție* (**threading**) cu cozi (`queue.Queue`), nu programare asincronă: apelurile de inferență ale modelelor sunt operații blocante, intensive computațional, care nu trebuie să blocheze bucla de evenimente a serverului FastAPI. Pipeline-ul folosește în total **șapte fire de execuție**: captură (Thread 1), recunoaștere facială cu demografie (Thread 2), plăcuțe (Thread 3), foc și fum (Thread 4), acțiuni umane (Thread 5), arme (Thread 6) și un fir final de fuziune (Thread 7).

5.7.1 Dispecerizarea cadrelor și procesarea în paralel

Firul de captură (Thread 1) citește cadre de la toate camerele active (camera laptop-ului și eventualele camere IP). Fiecărui cadru i se atribuie un identificator unic și monoton crescător, `frame_id`, care va servi drept cheie de sincronizare pe tot parcursul pipeline-ului. Cadrul este apoi *dispecerizat* de metoda `_dispatch_frame()`, care îi pune câte o copie în coada de intrare a fiecărui modul activat (`face_processing_queue`, `plate_processing_queue` etc.) și, separat, în coada de cadre brute (`raw_frame_queue`), care păstrează imaginea originală pentru recompunere.

Fiecare modul AI rulează în propriul fir, consumând cadre din coada sa, executând inferența și publicând rezultatele în coada sa de ieșire (`face_results_queue` etc.). Cum cozile au capacitate mărginită (`maxsize=10`), atunci când un modul lent nu mai poate prelua cadre, dispecerul nu blochează, ci sare peste cadru (incrementând un contor de `queue_skips` / `frames_dropped`). Astfel, un model mai lent degradează elegant rata de cadre, fără a opri întregul sistem.

5.7.2 Sincronizarea rezultatelor după `frame_id`

Firul de fuziune (Thread 7) are sarcina dificilă de a recombina rezultate care sosesc *asincron*: modulul de plăcuțe poate termina cadrul N înainte ca modulul facial să termine cadrul $N - 2$. Pentru aceasta, firul menține un dicționar `pending_results`, cu câte o intrare pentru fiecare tip de rezultat (`frames`, `face`, `plate`, `fire`, `har`, `weapon`), indexată după `frame_id`. La fiecare iterație, firul golește toate cozile de rezultate disponibile în acest dicționar, apoi încearcă să *asambleze* fiecare cadru pentru care a sosit imaginea brută.

Un cadru este considerat gata de asamblare atunci când rezultatele modulelor *esențiale* activate (recunoaștere facială și plăcuțe) au sosit. Rezultatele modulelor de foc, acțiuni și arme sunt tratate ca *opționale*: ele sunt suprapuse dacă au sosit, dar absența lor nu blochează asamblarea. Pentru a evita blocajul atunci când un rezultat nu mai sosește (de exemplu, din cauza unui cadru sărit), există un mecanism de *timeout*: dacă un cadru devine prea vechi (diferența dintre `frame_id`-ul curent și al lui depășește 60), el este asamblat cu rezultatele disponibile la acel moment, fără a mai aștepta. În plus, o rutină de curățare (`_cleanup_old_results()`) elimină periodic intrările mai vechi de 90 de cadre, pentru ca dicționarul să nu crească nelimitat în caz de desincronizare.

5.7.3 Compunerea cadrului adnotat

Odată ce un cadru este gata de asamblare, adnotările celor cinci module sunt desenate *în straturi* peste imagine, într-o ordine fixă. Punctul de plecare este cadrul deja adnotat de modulul facial (care conține chenarele fețelor și etichetele demografice), peste care se aplică succesiv: chenarele plăcuțelor (`plate_system.draw_outputs()`), detecțiile de foc confirmate (`fire_system.draw_detections()`), eticheta acțiunii recunoscute (`har_system.draw_detections()`) și, la final, chenarele armelor (`weapon_system.draw_detections()`). Rezultatul este un singur cadru (`final_frame`)

care concentrează vizual toate detecțiile celor cinci module pentru acel moment de timp.

5.7.4 Generarea alarmelor și deduplicarea

În aceeași etapă de fuziune, detecțiile relevante sunt transformate în două tipuri de înregistrări persistente: *jurnale de detecție* (un istoric complet, scris prin `_log_detection_if_needed()`) și *alarme* propriu-zise (evenimente care necesită atenția operatorului). Sunt considerate demne de alarmă: persoanele necunoscute, focul confirmat cu `alert`, acțiunile `non-normal`, armele detectate, vehiculele neautorizate (plăcuțe lizibile care nu sunt pe lista albă) și prezența unei persoane neautorizate într-o zonă restricționată.

Problema centrală a generării alarmelor este evitarea inundării operatorului cu duplicate: un foc care arde 10 secunde nu trebuie să genereze o alarmă la fiecare cadru. Mecanismul de deduplicare, implementat în `_create_alarm_if_needed()`, folosește o cheie compusă din cameră, tip și *subiect* (`camera_id:alarm_type:dedup_subject`) și un cooldown de 30 de secunde per cheie. Componenta de subiect este esențială: ea permite ca alarme distincte de *același* tip să coexiste – de exemplu, două persoane necunoscute diferite (diferențiate prin poziție), un cuțit și un pistol, sau două plăcuțe neautorizate diferite – fără a se contopi într-o singură alarmă. Pe lângă bariera din memorie, există și o verificare de rezervă în baza de date (`get_recent_alarm()`), care previne avalanșa de alarme duplicate imediat după o repornire a serverului, când starea de cooldown din memorie este goală.

5.7.5 Difuzarea către clienți prin WebSocket

Ultimul pas al fuziunii este livrarea cadrului asamblat către interfețele clienților. Cadrul `final_frame` este redimensionat (la maximum 800 px lățime), codat JPEG cu o calitate de 75 și convertit în Base64 prin `_encode_and_queue_frame()`. Acesta este împachetat într-un mesaj JSON care conține, pe lângă imagine, listele de rezultate serializate ale tuturor modulelor (`face_results`, `plate_results`, `fire_results`, `har_results`, `weapon_results`), identificatorul camerei, un *timestamp* și starea (activat/dezactivat) a fiecărui modul.

Difuzarea propriu-zisă (`_broadcast_frame()`) folosește un model de tip *fan-out* cu cozi independente per client: fiecare client conectat prin WebSocket are propria coadă de ieșire, iar firul de fuziune publică fiecare cadru procesat în toate cozile. Această izolare este importantă – un client lent își aruncă doar propriul cadru cel mai vechi atunci când coada lui se umple, fără a “fura” cadre de la ceilalți clienți sau a-i încetini. Astfel, fluxul video adnotat ajunge la fiecare operator în mod independent, încheind ciclul de procesare care a pornit de la captura cadrului brut.

NECLASIFICAT

Capitolul 6: Implementarea Aplicației Web

- 6.1 Structura backend-ului FastAPI
- 6.2 Endpoint-ul WebSocket pentru streaming live
- 6.3 Modelul de date și persistența
- 6.4 Sistemul de autentificare
- 6.5 Modulele frontend-ului React
 - 6.5.1 Autentificare
 - 6.5.2 Vizualizarea live a camerelor (Dashboard)
 - 6.5.3 Activarea/Dezactivarea modulelor AI
 - 6.5.4 Gestiunea persoanelor (CRUD)
 - 6.5.5 Gestiunea zonelor
 - 6.5.6 Gestiunea camerelor
 - 6.5.7 Gestiunea plăcuțelor de înmatriculare
 - 6.5.8 Gestiunea alarmelor
 - 6.5.9 Jurnale de activitate
 - 6.5.10 Statistici și KPI-uri
 - 6.5.11 Gestiunea utilizatorilor
- 6.6 Logica de business pentru declanșarea alertelor
- 6.7 Sincronizarea camerei conectate dinamic cu registrul de camere

Capitolul 7: Testarea și Evaluarea Sistemului

7.1 Metodologia de testare

7.2 Configurația hardware folosită

7.3 Evaluarea pipeline-ului integrat

7.3.1 FPS la procesare concurentă

7.3.2 Utilizarea memoriei GPU și RAM

7.3.3 Latența end-to-end

7.4 Scenarii de test funcțional

7.5 Discuție asupra false pozitivelor și false negative

7.6 Limitări observate

Capitolul 8: Concluzii și Direcții de Dezvoltare

- 8.1 Sinteza rezultatelor
- 8.2 Obiective atinse vs. obiective propuse
- 8.3 Contribuții originale
- 8.4 Limitări ale soluției actuale
- 8.5 Direcții de dezvoltare viitoare

Bibliografie

Cărți

Articole Științifice